Vision avancée et transfer learning

Jour 1 · Application au deep learning

ING3 option IA · CY Tech



Le module : six journées et un projet

Pourquoi : savoir ce que chaque journée apporte, et au projet, le seul travail évalué.

9h30 - 12h30

Cours

les modèles et leur code PyTorch, sur un cas d'étude

→

13h30 - 17h

TD

un notebook marimo sur CPU, toujours face à une méthode classique

→

en groupe

Projet Usine 4.0

la même démarche, sur les images d'une aciérie

Jour	Le but de la journée	Modèles et méthodes	Dans le projet
1	Reconnaître une image, avec peu d'exemples	CNN, ResNet, EfficientNet, ViT ; transfer learning	type de défaut : exigé
2	Trouver et délimiter les objets d'une image	YOLO, Faster R-CNN, U-Net, DeepLab, SAM	localisation : exigée
3	Prévoir une mesure, repérer une anomalie	LSTM, TCN, Transformer, autoencodeur	pompe : volet au choix
4	Classer sans exemple annoté ; générer des images	CLIP, DINO, modèles de diffusion	défauts rares : volet au choix
5	Données en réseau ; apprendre à décider	GCN, GAT ; DQN, PPO	hors projet
6	Mettre un modèle en service	quantification, export ONNX, distillation, suivi	cadence et suivi : exigés



Évaluation : le projet seul, en groupes de 3 ou 4 ; une note de groupe, sans oral ; rendu deux semaines après le Jour 6. Sujet : projet/SUJET.md, dans le dépôt github.com/FranklinB13/applications-modeles-ia-td.

Le cas d'étude de la séance

Pourquoi : toutes les notions du jour sont appliquées à ce cas, le matin comme au TD ; ses trois questions sont les objectifs de la journée.

ImageNet-1k

1,28 million d'images annotées, 1000 classes

pré-entraînement → déjà fait par d'autres

Réseau pré-entraîné

ResNet, EfficientNet, ViT : poids publics

adaptation → ce que l'on apprend aujourd'hui

Notre tâche

peu d'images annotées, une machine sans GPU

Reconnaître la race d'un chat ou d'un chien sur une photo

- **Données** : jeu Oxford-IIIT Pet, 37 races, 30 photos d'entraînement par race
- **Matériel** : les machines de l'école, sans carte graphique (GPU)
- **À rendre** : un modèle choisi parmi plusieurs, avec les mesures qui justifient ce choix

Les trois questions de la séance

1 **Quel modèle pré-entraîné choisir ?** — parties 2 à 4

comparer ResNet, EfficientNet et un Vision Transformer

2 **Comment l'adapter à nos 37 races ?** — parties 6 et 7

garder ses features (feature extraction) ou réentraîner une partie du réseau (fine-tuning)

3 **Comment vérifier qu'il fonctionne ?** — parties 5 et 8

le mesurer sans biais, face à une méthode classique, et comprendre ses erreurs

Source : Parkhi, Vedaldi, Zisserman et Jawahar, Cats and Dogs (jeu Oxford-IIIT Pet), CVPR 2012.

Plan de la matinée

Pourquoi : le cours suit l'ordre du TD ; sur chaque diapositive, la pastille orange « TD · étape N » dit où elle sert.

9h30 · avant la pause

Partie 1

Ce qu'est un CNN

convolution, pooling, BatchNorm

les bases de ResNet et EfficientNet

Partie 2 — 1

ResNet

les connexions résiduelles

TD · étape 2

Partie 3 — 1

EfficientNet

agrandir au meilleur coût

TD · étape 2

Partie 4 — 1

Vision Transformer

l'attention, appliquée aux images

TD · étape 2

après la pause

Partie 5 — 3

Comparer proprement

validation, test, baseline

TD · étapes 3 et 5

Partie 6 — 2

Transfer learning

la feature extraction

TD · étape 4

Partie 7 — 2

Fine-tuning

réentraîner les derniers étages

TD · étape 6

Partie 8 — 3

Évaluation et diagnostic

matrice de confusion, Grad-CAM

TD · étapes 7 et 8

Supposé connu : Python et PyTorch (tenseurs, nn.Module, boucle d'entraînement), réseau dense et descente de gradient ; CNN et attention dans les grandes lignes, rappelés en parties 1 et 4.



Partie 1

Ce qu'est un CNN : convolution, pooling, BatchNorm

Pourquoi : lire l'architecture des candidats, et estimer ce qu'elle coûte

Les briques d'un CNN : définition

Définition

Un réseau de neurones convolutif (CNN) enchaîne trois briques : la convolution, qui fait glisser de petits filtres appris sur l'image ; le pooling, qui résume chaque petite zone par un nombre ; la normalisation par lot (BatchNorm), qui garde stable l'échelle des valeurs.

Le problème que cela résout

Une couche dense reliée à tous les pixels aurait des millions de poids. La convolution réutilise le même petit filtre partout : très peu de poids, et le même motif est détecté où qu'il soit.

Ce qui entre, ce qui sort

Entrée : une image (canaux × hauteur × largeur). Sortie : des cartes de features, puis un score par classe.

Ce que l'on va voir, dans l'ordre

2. Vue d'ensemble : ce qu'est un CNN
3. La convolution
4. Un filtre sur une vraie image
5. Convolution sur plusieurs canaux
6. Taille de la sortie et nombre de poids
7. Le pooling
8. Convolution et couche dense : nombre de paramètres
9. Le champ récepteur
10. La normalisation par lot
11. BatchNorm : entraînement et évaluation
12. À retenir

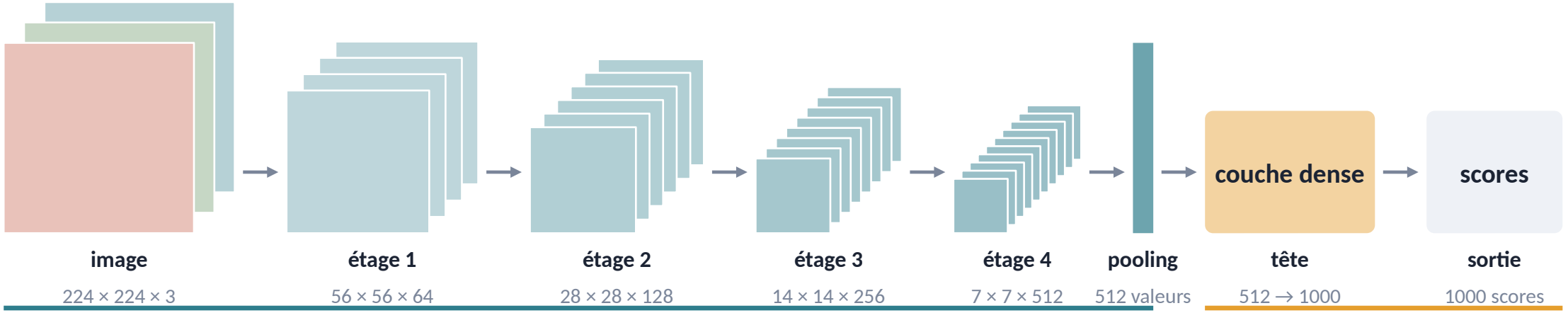
Dans le TD : ResNet-18 et EfficientNet-B0, deux des trois candidats, sont des CNN.

Vue d'ensemble : ce qu'est un CNN

Pourquoi : le schéma complet, avant d'en détailler les briques, dans l'ordre.

Un **CNN** a deux parties. Le **backbone** transforme l'image en **cartes** de plus en plus petites et nombreuses, qui disent où sont les motifs. La **tête** en déduit un score par classe.

Un étage : un groupe de couches qui travaillent à la même taille. D'un étage au suivant, la carte devient deux fois plus petite et le nombre de canaux double.



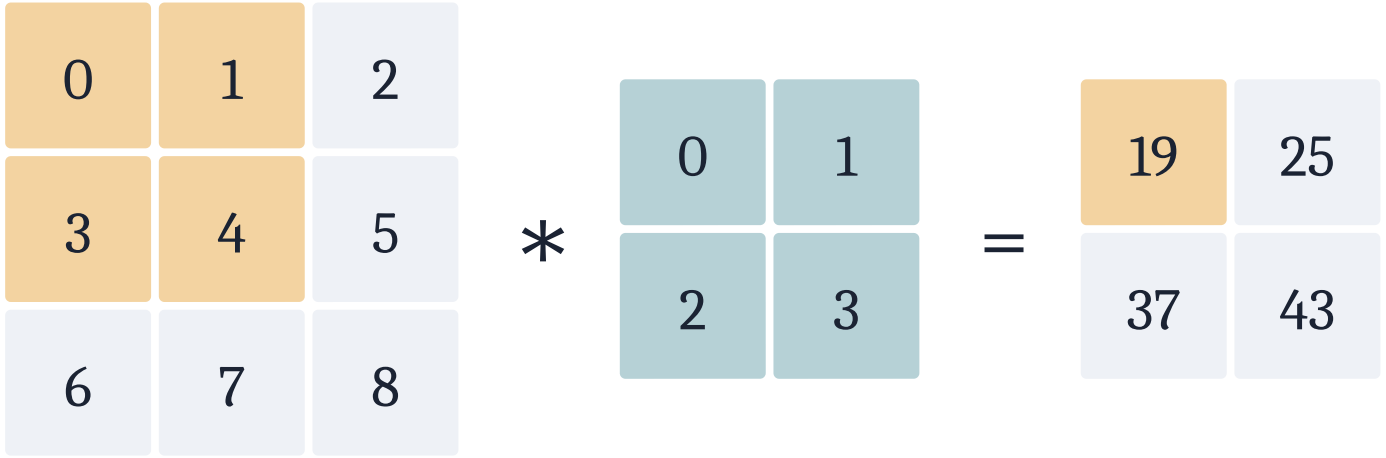
- Ce schéma est **ResNet-18**, le premier candidat : dans torchvision, ses étages s'appellent **layer1** à **layer4** et sa tête **fc**. Au TD, on garde le backbone et on remplace la tête
- Le troisième candidat, le **Vision Transformer**, n'est pas un CNN : il remplace les convolutions par de l'**attention** (partie 4)

Dimensions de ResNet-18 (torchvision), relevées par une passe avant. Même schéma : CS231n, Architecture Overview ; d2l, sections 7.6 (LeNet) et 8.8 (stem, étages, tête).

La convolution

Pourquoi : c'est l'opération de base d'un CNN : on la calcule d'abord à la main.

Une convolution fait glisser un petit **noyau** de poids sur l'image ; en chaque position, elle multiplie case à case et additionne. Le résultat est une **carte**.



entrée I — noyau K — sortie S

- **En haut à gauche** : 0 × 0 + 1 × 1 + 3 × 2 + 4 × 3 = **19**
- On décale le noyau d'une case : **25** ; une ligne plus bas : **37**, puis **43**
- La sortie est plus petite que l'entrée : le noyau ne déborde pas

$$S(i,j) = \sum_m \sum_n K(m,n) \cdot I(i+m, j+n)$$

Exemple de d2l, figure 7.2.1.

Un filtre sur une vraie image

Pourquoi : voir ce que détecte un filtre ; dans un CNN, les filtres ne sont pas choisis, ils sont appris.

Le filtre de Sobel x

−1	0	1
−2	0	2
−1	0	1

Il réagit aux variations horizontales d'intensité, donc aux **bords verticaux** : contours de l'œil, moustaches.

Image d'entrée (niveaux de gris)



Sortie du filtre de Sobel (valeur absolue)



Image : scikit-image (chelsea). Sortie calculée avec le filtre ci-contre.

Traitement d'image classique : filtres **choisis à la main**. CNN : filtres **appris** à partir des données.

Convolution sur plusieurs canaux

Pourquoi : c'est ce que calcule vraiment une couche de CNN ; il faut le savoir pour lire le code et estimer le coût d'un réseau.

Une image en couleur a **3 canaux** (rouge, vert, bleu). Un filtre regarde les 3 à la fois et produit **une seule carte**. Une couche a plusieurs filtres, donc plusieurs cartes.



l'image : 3 canaux

un filtre : un carré 3 × 3 par canal

une carte

64 cartes : les 64 canaux de la couche suivante

Au milieu d'un réseau, c'est pareil : seulement plus de canaux (64, 128, 512) au lieu de 3.

Source : d2l, section 7.4 (canaux) ; CS231n, Convolutional Networks.

Taille de la sortie et nombre de poids

Pourquoi : deux calculs à savoir faire pour lire une architecture et estimer son coût.

Deux calculs à savoir faire pour lire une architecture : la **taille** de la carte produite, et le **nombre de poids** de la couche.

La taille de la carte

- Le **pas** : de combien de pixels le filtre avance. Pas de 2 : la carte est **deux fois plus petite**
- Le **padding** : des zéros ajoutés autour de l'image, pour que le filtre puisse se poser sur les bords
- Exemple : la première couche de ResNet (pas de 2) passe de **224 à 112** pixels

Le nombre de poids

- Un filtre 3 × 3 sur 3 canaux : 3 × 3 × 3 = **27 poids**, plus 1 biais
- 64 filtres : 64 × 28 = **1 792 poids**
- Il ne dépend **pas** de la taille de l'image : le même filtre sert partout

Formule de la taille, pour les curieux : partie entière de (H + 2p − k) / s, plus 1 (H : taille de l'entrée, k : taille du filtre, p : padding, s : pas).

Source : d2l, sections 7.3 (padding et pas) et 7.4 (canaux).

Le pooling

Pourquoi : c'est la deuxième opération d'un CNN : elle réduit la taille des cartes ; ResNet-18 en a une à l'entrée et une avant sa tête.

Le pooling découpe chaque carte en petites zones et résume chaque zone par **un seul nombre** : son **maximum** (max pooling) ou sa **moyenne** (average pooling). Il n'a **aucun paramètre** et traite chaque canal séparément.

Une carte 4 × 4

1	3	2	0
5	4	1	1
0	2	6	7
1	1	8	3

Max pooling 2 × 2

5	2
2	8

Average pooling 2 × 2

3,25	1
1	6

Chaque couleur : une zone 2 × 2 et la valeur qui la résume. Pas de 2 : la carte passe de 4 × 4 à 2 × 2.

À quoi il sert

- **Réduire la résolution** : moins de calcul pour les couches suivantes, et un champ récepteur qui grandit plus vite
- **Tolérer un petit décalage** : un motif déplacé d'un pixel donne souvent le même maximum

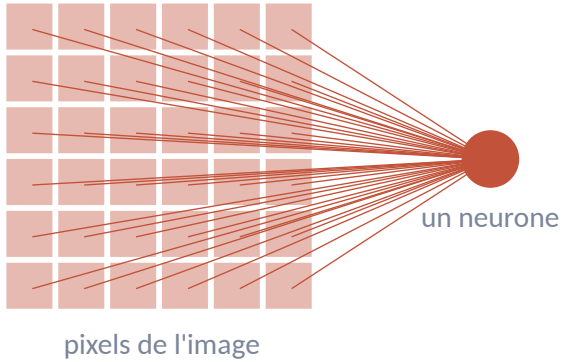
Pooling global : une seule zone, la carte entière ; il reste une valeur par canal. C'est ainsi que ResNet-18 passe de 512 × 7 × 7 à 512 valeurs avant sa tête.

Source : d2l, section 7.5 (Pooling) ; CS231n, Pooling Layer.

Convolution et couche dense : nombre de paramètres

Pourquoi : comprendre pourquoi un CNN a besoin de moins de données qu'un réseau dense, ou qu'un ViT.

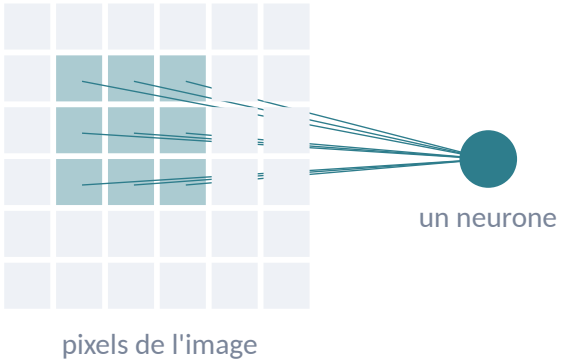
Couche dense : chaque neurone est relié à tous les pixels



150,5 M

poids pour relier une image 224 × 224 × 3 à 1000 neurones

Convolution : un neurone ne voit qu'une zone 3 × 3



1 728

poids pour 64 filtres 3 × 3 sur 3 canaux, réutilisés à chaque position

Connectivité locale

chaque neurone ne regarde qu'un petit voisinage

Partage des poids

le même filtre sert à toutes les positions

Même réponse partout

si le chat se déplace dans l'image, sa trace se déplace de la même façon dans la carte

Calculs sans les biais. Ces trois propriétés sont des a priori sur les images : le ViT en a beaucoup moins.

Le champ récepteur

Pourquoi : c'est ce qui permet à un empilement de petits filtres de reconnaître un objet entier.

Le **champ récepteur** d'un neurone est la zone de l'image dont il dépend. Il grandit à chaque couche : c'est ainsi qu'un empilement de filtres 3 × 3 finit par voir un objet entier.

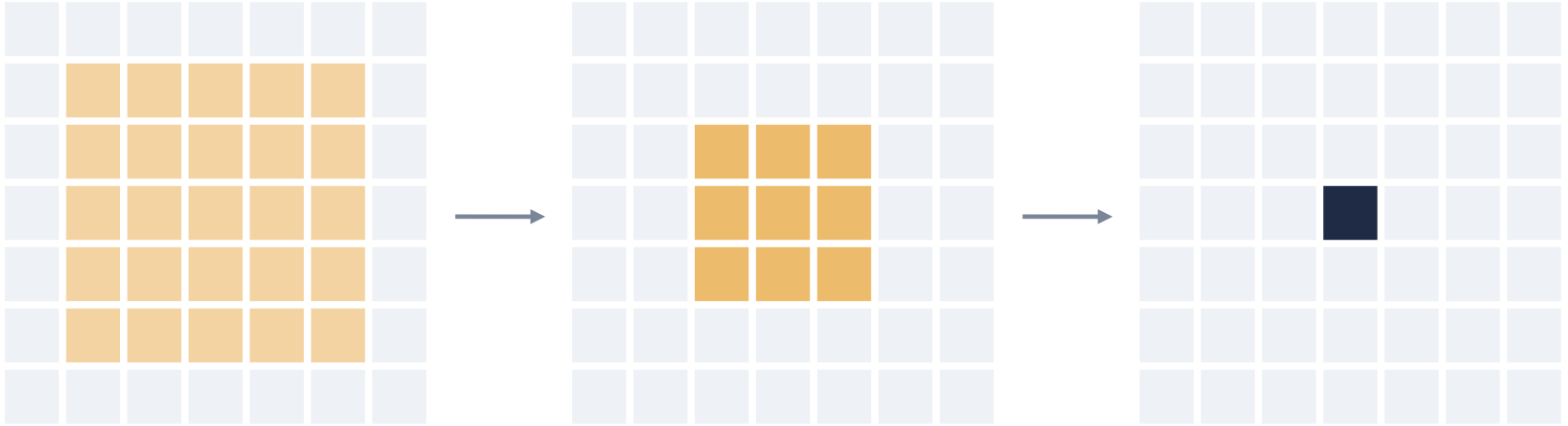


image : zone 5 × 5 | couche 1 : zone 3 × 3 | couche 2 : un neurone

- Une convolution 3 × 3 : le neurone voit une zone **3 × 3** de la couche précédente
- Deux convolutions empilées : **5 × 5** de l'image ; trois : **7 × 7**
- Pooling et pas de 2 : la résolution baisse, le champ grandit plus vite

Les premières couches réagissent à des **bords**, les suivantes à des **textures**, puis à des **parties d'objets**, enfin à des **objets** entiers.

Source : d2l, section 7.2.6 (Feature Map and Receptive Field) ; CS231n, Convolutional Networks.

La normalisation par lot

Pourquoi : elle suit les convolutions de ResNet et d'EfficientNet ; on la calcule ici sur un exemple.

La **BatchNorm** remet les valeurs de chaque canal à une échelle standard, calculée sur le lot d'images en cours. Un exemple : un canal, un lot de 4 valeurs.

1	**Le lot**	2	4	6	8	moyenne 5 ; écart-type 2,24
2	**Centrer, réduire**	−1,34	−0,45	0,45	1,34	on retire 5, on divise par 2,24 : moyenne 0, écart-type 1
3	**Facteur, décalage**	−1,68	0,11	1,89	3,68	on multiplie par γ = 2, on ajoute β = 1 (deux nombres appris)

Pour chaque canal : on **centre et réduit** sur le lot, puis le réseau choisit lui-même l'échelle avec **deux nombres appris**, γ et β.

Valeurs de l'exemple calculées avec nn.BatchNorm1d. Ioffe et Szegedy, Batch Normalization, ICML 2015 ; d2l, section 8.5.

BatchNorm : entraînement et évaluation

Pourquoi : c'est un piège du fine-tuning, à l'étape 6 du TD.

La BatchNorm ne fait pas le même calcul pendant l'entraînement et pendant la prédiction : il faut lui dire dans quel mode elle est.

En entraînement : **model.train()**

μ et σ sont ceux du **lot en cours** ; une moyenne mobile de ces statistiques est mise à jour à chaque lot.

En évaluation : **model.eval()**

μ et σ sont les **statistiques mémorisées** : la prédiction d'une image ne dépend plus des autres images du lot.

Toujours **model.eval()** avant de prédire : sinon, l'image est normalisée avec les statistiques du lot de test, et sa prédiction dépend des autres images.

Source : Documentation PyTorch : torch.nn.BatchNorm2d ; le code est en annexe (BatchNorm en PyTorch).

Les briques d'un CNN : à retenir

1. Une **convolution** fait glisser des filtres appris ; chaque filtre couvre tous les canaux d'entrée et produit une carte. Le même filtre sert partout : très peu de poids.

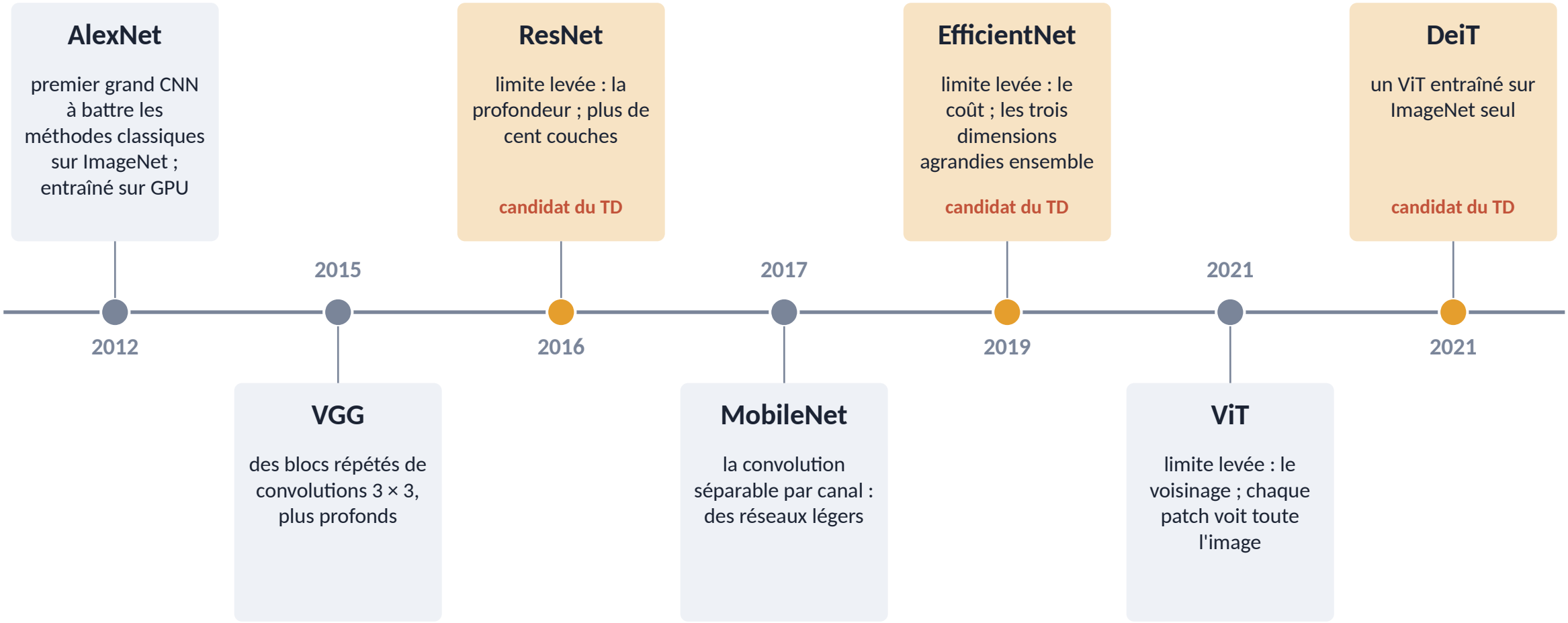
2. Le **pooling** résume chaque zone par son maximum ou sa moyenne, sans paramètre ; le pooling global donne une valeur par canal. Couche après couche, le **champ récepteur** grandit.

3. La **BatchNorm** centre et réduit chaque canal sur le lot, puis applique un facteur et un décalage appris. Elle se comporte différemment en **train()** et en **eval()** : toujours **model.eval()** avant de prédire.

Pour approfondir : d2l, chapitre 7 : convolution, canaux, pooling ; d2l, section 8.5 : Batch Normalization

D'AlexNet à DeiT : situer les modèles du jour

Pourquoi : les trois candidats sont des étapes d'une même histoire ; chacune répond à une limite de la précédente.



Années : publication des articles. Ordre et rôle d'ImageNet comme « baromètre » : d2l, introduction du chapitre 8. Candidats : ResNet-18, EfficientNet-B0, DeiT-tiny.



Partie 2

ResNet et les connexions résiduelles

Premier candidat : le réseau de référence, et l'idée qui permet d'entraîner des réseaux très profonds

ResNet : définition

Définition

ResNet est une famille de CNN très profonds, de 18 à 152 couches, dont chaque bloc ajoute son entrée à sa sortie : c'est la connexion résiduelle.

Le problème que cela résout

Au-delà d'une certaine profondeur, ajouter des couches à un CNN classique fait monter son erreur, même sur les données d'entraînement. La connexion résiduelle lève cette difficulté.

Ce qui entre, ce qui sort

Entrée : une image 224 × 224. Sortie : un score par classe (les 1000 classes d'ImageNet).

Ce que l'on va voir, dans l'ordre

2. Le problème de la dégradation

3. Réponse : imiter un réseau moins profond

4. Le bloc résiduel

5. Charger un ResNet-18 pré-entraîné

6. À retenir

Dans le TD : ResNet-18 est le premier candidat, aux étapes 2 et 4.

Le problème de la dégradation

Pourquoi : c'est le problème que ResNet résout ; sans cette solution, pas de réseaux de plus de cent couches.

Réseau « simple » de 20 couches



erreur d'entraînement plus faible

Réseau « simple » de 56 couches



erreur d'entraînement plus élevée (et erreur de test aussi)

Un rectangle = une couche. Réseaux « simples » : couches empilées, avec BatchNorm, sans raccourci.

Sur CIFAR-10, un réseau « simple » de 56 couches fait moins bien qu'un réseau de 20 couches, **y compris sur les données d'entraînement**.

Ce n'est pas du surapprentissage : c'est l'**optimisation** qui échoue. On parle de **dégradation**.

He et al., Deep Residual Learning for Image Recognition, CVPR 2016, figure 1.

Question à la salle

Un réseau de 56 couches pourrait au moins imiter celui de 20 couches. Comment ?



Réponse : imiter un réseau moins profond

Pourquoi : cet argument mène directement à l'idée de ResNet.

20 couches : copiées du petit réseau **36 couches : elles calculent l'identité, sortie = entrée**



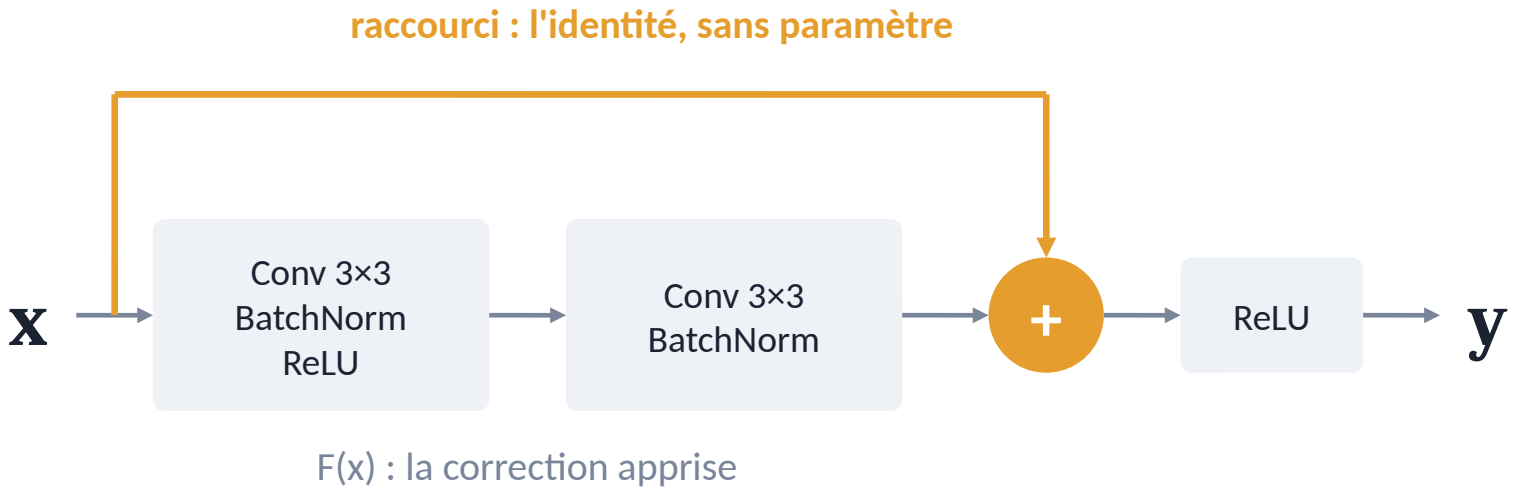
- Le réseau de 56 couches fait alors **exactement** comme celui de 20 : une solution au moins aussi bonne existe. Pourquoi l'entraînement ne la trouve-t-il pas ? Hypothèse des auteurs : l'optimiseur a du mal à approcher l'identité avec une pile de couches non linéaires (convolutions, BatchNorm, ReLU)
- D'où l'idée de ResNet, sur la diapositive suivante : rendre l'identité **facile à apprendre**

La solution existe ; c'est la **descente de gradient** qui ne la trouve pas : un problème d'optimisation, pas de capacité.

Source : He et al., Deep Residual Learning for Image Recognition, CVPR 2016, section 3.1.

Le bloc résiduel

Pourquoi : c'est l'élément de base de ResNet ; on le retrouve dans EfficientNet et dans les Transformers.



F(x) : la correction apprise

$$y = \mathrm{ReLU}(F(x) + x)$$

- Si l'identité est la bonne réponse, il suffit que **F soit nulle** : facile à apprendre
- Le bloc apprend une **correction** de son entrée, pas une transformation complète

Source : He et al., CVPR 2016, section 3.1 et figure 2 ; d2l, section 8.6 (ResNet).

Charger un ResNet-18 pré-entraîné

Pourquoi : c'est le point de départ de l'étape 2 du TD.

```python
import torch
from PIL import Image
from torchvision.models import resnet18, ResNet18_Weights

poids = ResNet18_Weights.IMAGENET1K_V1
modele = resnet18(weights=poids).eval()
pretraitement = poids.transforms()   # le prétraitement d'évaluation

print([nom for nom, _ in modele.named_children()])
# ['conv1', 'bn1', 'relu', 'maxpool', 'layer1', 'layer2',
#  'layer3', 'layer4', 'avgpool', 'fc']
print(modele.fc)   # Linear(in_features=512, out_features=1000, bias=True)

image = Image.open("chat.jpg").convert("RGB")
x = pretraitement(image).unsqueeze(0)   # (1, 3, 224, 224)
with torch.no_grad():
    probas = modele(x).softmax(dim=1)
valeurs, indices = probas.topk(3)
# les 3 classes les plus probables, par leur nom
print([poids.meta["categories"][i] for i in indices[0]])
```

- **weights=** télécharge les poids la première fois, puis les garde en cache
- **poids.transforms()** : le prétraitement d'évaluation associé à ces poids (redimensionnement, recadrage central, normalisation)
- **.eval()** et **torch.no_grad()** pour prédire
- La tête s'appelle **fc** : 512 entrées, 1000 classes

Tous les extraits de code du cours ont été exécutés avec les versions de uv.lock (PyTorch 2.14.1) : les sorties écrites en commentaire sont celles obtenues.

Source : Documentation torchvision : resnet18 et ses poids IMAGENET1K_V1.

ResNet : à retenir

1. Un réseau « simple » trop profond s'entraîne moins bien, **même sur les données d'entraînement** : c'est la dégradation, un problème d'optimisation.

2. Le bloc résiduel calcule **x + F(x)** : l'identité correspond à F = 0, facile à apprendre. On peut alors empiler plus de cent couches.

3. ResNet-18 suit le schéma de la partie 1 : quatre étages, **layer1** à **layer4**, un pooling global, puis une **tête fc** 512 → 1000, que l'on remplacera.

Pour approfondir : d2l, section 8.6 : ResNet



Partie 3

EfficientNet et le compound scaling

Deuxième candidat : le plus économe en calcul, celui que l'on fine-tune au TD

EfficientNet : définition

Définition

EfficientNet est une famille de CNN, de B0 à B7, obtenue en agrandissant un petit réseau de base, B0, à la fois en profondeur, en largeur et en résolution, selon une règle fixe.

Le problème que cela résout

Agrandir un réseau au hasard coûte cher pour peu de gain. EfficientNet vise la meilleure accuracy pour un budget de calcul donné.

Ce qui entre, ce qui sort

Entrée : une image (224 × 224 pour B0). Sortie : un score par classe.

Ce que l'on va voir, dans l'ordre

2. Trois façons d'agrandir un réseau
3. Compound scaling
4. Convolution séparable par canal
5. Accuracy et nombre de paramètres
6. Mesurer un temps d'inférence
7. À retenir

Dans le TD : EfficientNet-B0 est le deuxième candidat, et le modèle fine-tuné à l'étape 6.

Trois façons d'agrandir un réseau

Pourquoi : ce sont les trois leviers pour rendre un réseau plus précis, et plus coûteux.



Barres : les couches (longueur : nombre de canaux). Carré orange : l'image d'entrée.

Agrandir **une seule** dimension rapporte de moins en moins ; les trois sont liées : une image plus grande demande plus de couches et plus de canaux.

Source : Tan et Le, EfficientNet, ICML 2019, figures 2 et 3.

Question à la salle

On double la largeur d'un réseau. Par combien multiplie-t-on le calcul d'une convolution ?



Compound scaling

Pourquoi : c'est la règle qui agrandit les trois dimensions ensemble : l'idée d'EfficientNet.

EfficientNet agrandit le réseau **un peu dans les trois dimensions à la fois**, toujours dans les mêmes proportions. C'est le **compound scaling**.

À chaque cran, de B0 à B1, de B1 à B2, et ainsi de suite jusqu'à B7 :

Couches	Canaux	Taille de l'image
× 1,2	× 1,1	× 1,15
plus profond	plus large	plus de pixels

Les canaux et la taille de l'image comptent **au carré** dans le calcul (question précédente) : $1{,}2 \times 1{,}1^2 \times 1{,}15^2 \approx \mathbf{2}$.



Résultat : **le calcul double environ à chaque cran**. On choisit la taille, de B0 à B7, selon le budget de calcul.

Tan et Le, EfficientNet, ICML 2019, section 3.3 (dans l'article : α = 1,2 ; β = 1,1 ; γ = 1,15 ; φ, le nombre de crans).

Convolution séparable par canal

Pourquoi : c'est ce qui rend EfficientNet léger : moins de poids et moins d'opérations par image.

Une convolution classique fait deux travaux à la fois : regarder les pixels voisins et mélanger les canaux. La version **séparable** les fait **l'un après l'autre** : c'est presque 9 fois moins de poids.

Classique



589 824

poids en classique (256 canaux vers 256)

Séparable



67 840

poids en séparable

C'est ce qui rend **EfficientNet léger**. Mais moins de poids ne veut pas toujours dire plus rapide : le temps se **mesure** (étape 2 du TD).

Nombres de poids sans les biais, vérifiés avec PyTorch. En anglais : depthwise separable convolution. Le bloc MBConv d'EfficientNet est détaillé en annexe.

Accuracy et nombre de paramètres

Pourquoi : comparer les candidats sur ce qui compte pour nous : l'accuracy et le coût.



5,3 M

paramètres et 77,7 % pour EfficientNet-B0, contre 25,6 M et 76,1 % pour ResNet-50

0,39

milliard d'opérations par image pour EfficientNet-B0, contre 4,1 pour ResNet-50

EfficientNet-B0, deuxième candidat : à taille comparable, la meilleure accuracy ImageNet des trois, pour le moins d'opérations par image. Le temps réel, lui, se mesure (diapositive suivante).

torchvision (IMAGENET1K_V1, une vue), sauf DeiT-tiny : dépôt DeiT. Paramètres recomptés. EfficientNet-B7 est évalué en 600 × 600 pixels.

Mesurer un temps d'inférence

Pourquoi : le coût réel se mesure sur la machine : c'est l'étape 2 du TD.

```python
import time
import torch
from torchvision.models import efficientnet_b0

modele = efficientnet_b0().eval()   # poids aléatoires : suffit ici
x = torch.randn(1, 3, 224, 224)

with torch.no_grad():
    for _ in range(3):          # échauffement : premières passes plus lentes
        modele(x)
    debut = time.perf_counter()
    for _ in range(20):
        modele(x)
ms = (time.perf_counter() - debut) / 20 * 1000
print(f"{ms:.1f} ms par image, {torch.get_num_threads()} threads")
```

- **Échauffement** : les premières passes allouent la mémoire, elles sont plus lentes
- **Moyenne sur plusieurs passes**, jamais une mesure isolée
- **torch.no_grad()** : pas de graphe de calcul à l'inférence
- Le temps dépend de la machine et du nombre de threads : toujours les indiquer
- Pourquoi mesurer : le nombre d'opérations ne prédit pas le temps réel. Les trois candidats sont mesurés à l'étape 2 du TD

Source : Documentation Python : time.perf_counter ; documentation PyTorch : torch.no_grad.

EfficientNet : à retenir

1. On agrandit un réseau en **profondeur**, en **largeur** ou en **résolution** ; agrandir une seule dimension rapporte de moins en moins.

2. Le **compound scaling** les agrandit un peu toutes les trois à la fois : le calcul double environ à chaque cran, de B0 à B7.

3. Ses blocs **MBConv** (détaillés en annexe) utilisent la convolution séparable par canal, bien moins coûteuse. Le coût réel se **mesure** sur la machine cible.

Pour approfondir : Tan et Le, 2019 : l'article EfficientNet (arXiv)



Partie 4

Vision Transformer (ViT)

Troisième candidat : l'attention, puis le Transformer appliqué aux images, et ce qu'il demande en données

Le Vision Transformer : définition

Définition

Le Vision Transformer (ViT) découpe l'image en petits carrés, les patches, les traite comme les mots d'une phrase, et les fait passer dans un Transformer : une pile de blocs fondés sur l'attention, où chaque patch regarde tous les autres.

Le problème que cela résout

Une convolution ne voit que son voisinage : le contexte global n'arrive qu'après de nombreuses couches. L'attention relie toutes les zones de l'image dès la première couche ; en contrepartie, le ViT a moins d'a priori qu'un CNN et demande plus de données.

Ce qui entre, ce qui sort

Entrée : une image 224 × 224, découpée en 196 patches. Sortie : un score par classe.

Ce que l'on va voir, dans l'ordre

2. Le mécanisme d'attention
3. L'attention sur toute la séquence
4. Convolution et attention : comparaison
5. Découpage de l'image en patches
6. Architecture du ViT
7. Un bloc d'encodeur Transformer
8. Biais inductifs (a priori) et volume de données
9. DeiT : entraîner un ViT sur ImageNet seul
10. À retenir

Dans le TD : DeiT-tiny, un petit ViT, est le troisième candidat, aux étapes 2 et 4.

Le mécanisme d'attention

Pourquoi : le Vision Transformer, troisième modèle du jour, est construit sur l'attention.

Pour chaque élément d'une séquence, l'**attention** calcule une moyenne des **valeurs** de tous les éléments, lui compris. Le poids de chacun mesure la ressemblance entre la **requête** de l'élément et la **clé** de l'autre.

Requête q ce que l'élément cherche

Clé k ce que l'élément propose

Valeur v ce que l'élément transmet

Un exemple : la sortie de l'élément 1, dans une séquence de 4 éléments (d = 4 : dimension des requêtes et des clés)

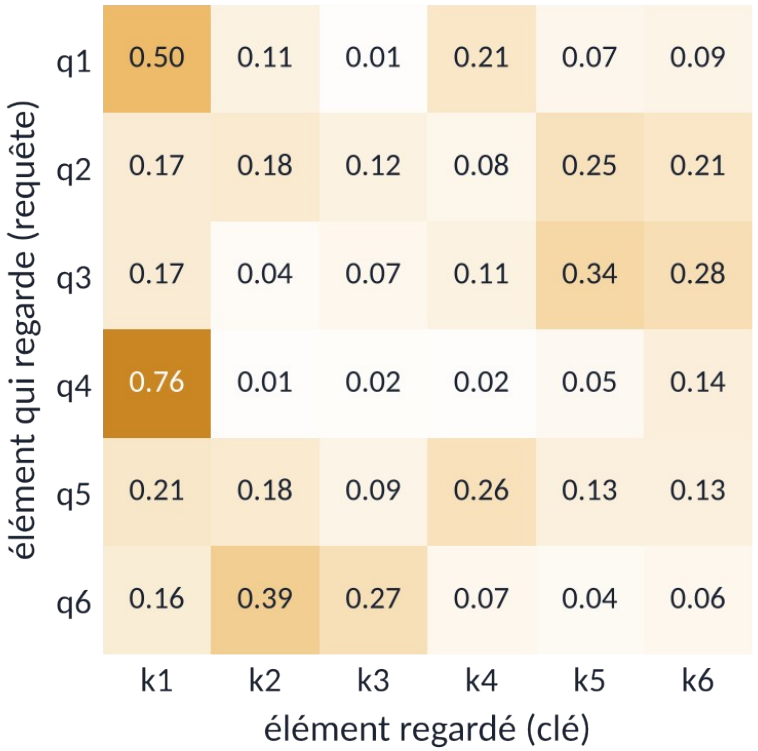
Élément j	Ressemblance avec l'élément 1	Divisée par √4 = 2	Poids (softmax : somme 1)	Valeur de j
1	4	2	0,53	(1 ; 0)
2	0	0	0,07	(0 ; 1)
3	2	1	0,20	(1 ; 1)
4	2	1	0,20	(0 ; 0)

Sortie de l'élément 1 = Σ poids × valeur = 0,53 × (1 ; 0) + 0,07 × (0 ; 1) + 0,20 × (1 ; 1) + 0,20 × (0 ; 0) = **(0,73 ; 0,27)**

Source : d2l, sections 11.1 et 11.3 ; Vaswani et al., Attention Is All You Need, NeurIPS 2017.

L'attention sur toute la séquence

Pourquoi : c'est le calcul que fait chaque couche du Vision Transformer, en une formule.



6 éléments (vecteurs aléatoires de dimension 8) : chaque ligne donne les poids d'un élément, et somme à 1.

Pour tous les éléments d'un coup, on range les requêtes, les clés et les valeurs dans trois tableaux, **Q**, **K** et **V**. Un seul calcul donne alors toutes les sorties :

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

- **QK^T** : toutes les ressemblances requête-clé ; le **softmax** en fait des poids de somme 1, ligne par ligne
- **$\div \sqrt{d_k}$** : une précaution, pour que les poids ne se concentrent pas tous sur un seul élément
- **Multi-têtes** : plusieurs attentions en parallèle, chacune regarde un aspect différent

Source : d2l, section 11.3 ; Vaswani et al., Attention Is All You Need, NeurIPS 2017.

Convolution et attention : comparaison

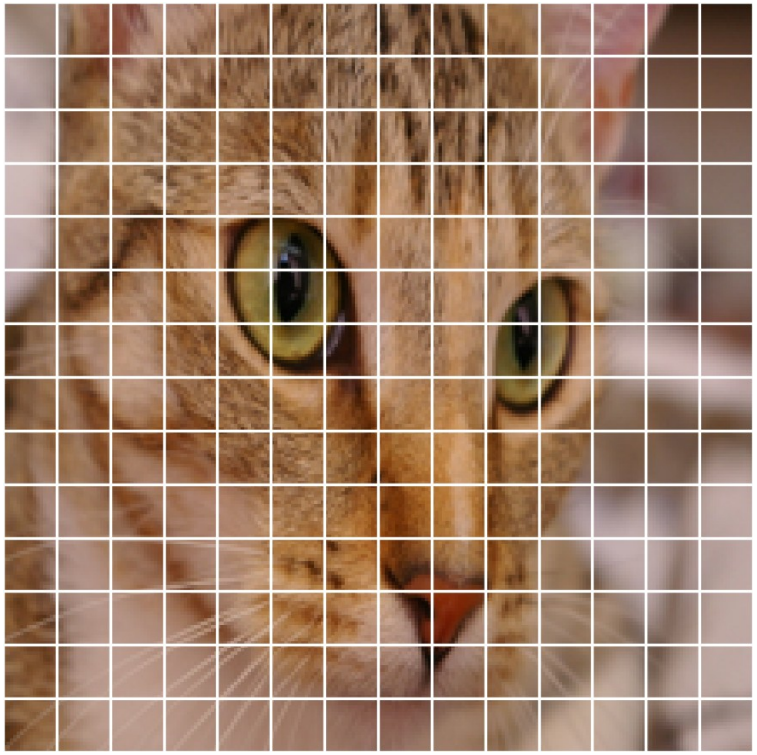
Pourquoi : ces différences expliquent quand un ViT convient mieux qu'un CNN, et inversement.

	Convolution	Attention
Portée	le voisinage du filtre	toute la séquence, dès la première couche
Poids utilisés	les poids du filtre, les mêmes partout	calculés pour chaque paire d'éléments, selon leur contenu
Coût pour N éléments	proportionnel à N	proportionnel à N^2
Ce qu'elle suppose d'avance	les pixels voisins vont ensemble ; un motif se reconnaît partout	rien : sans les positions, elle ne connaît même pas l'ordre des éléments

Source : d2l, section 11.6.2 : comparaison des CNN, des RNN et de l'attention (complexité, longueur des chemins).

Découpage de l'image en patches

Pourquoi : c'est ainsi qu'un Transformer, conçu pour du texte, peut lire une image.



1 **Découper**

image 224 × 224, patches de 16 × 16 : **14 × 14 = 196 patches**

2 **Aplatir**

chaque patch : 16 × 16 × 3 = **768 valeurs**

3 **Projeter**

une couche linéaire apprise en fait un **token de dimension D** : 192 pour DeiT-tiny, 768 pour ViT-Base



8 patches d'une même ligne : un extrait de la « phrase » de 196 patches que lit le Transformer

Source : Dosovitskiy et al., An Image is Worth 16x16 Words, ICLR 2021, section 3.1 ; d2l, section 11.8.

Architecture du ViT

Pourquoi : c'est le chemin d'une image dans le ViT, des patches aux scores.



Token [CLS]

un vecteur appris, placé en tête de la séquence ; en sortie, il résume toute l'image, et la tête ne lit que lui

Embedding de position

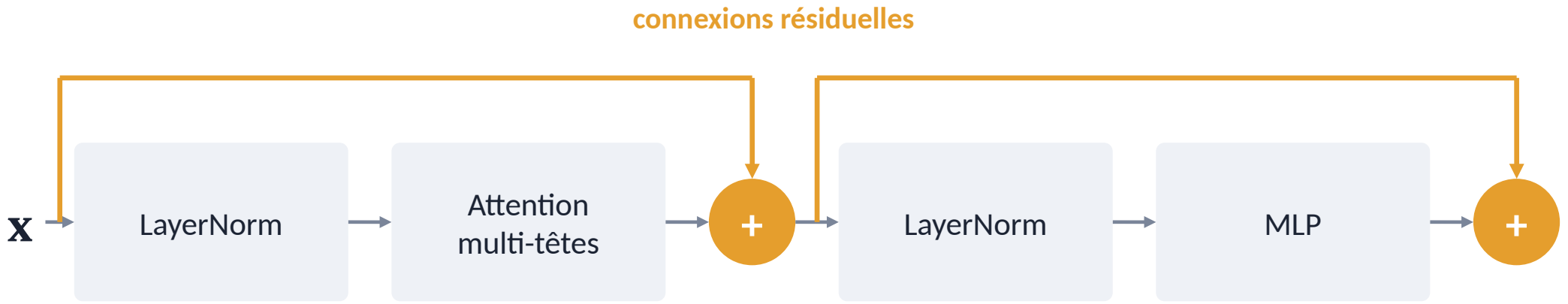
un vecteur appris, ajouté à chaque token ; sans lui, l'attention ne saurait pas où se trouve chaque patch

Au TD, on retire la tête de DeiT-tiny et on garde le vecteur **[CLS]** comme résumé de l'image : c'est sa feature extraction.

Source : Dosovitskiy et al., ICLR 2021, figure 1 et section 3.1 ; d2l, section 11.8.

Un bloc d'encodeur Transformer

Pourquoi : c'est le bloc que DeiT-tiny répète 12 fois ; on y retrouve l'idée de ResNet.



- **LayerNorm** : normalise chaque token sur ses propres valeurs, pas sur le lot : même calcul en entraînement et en évaluation
- **MLP** : deux couches denses, appliquées à chaque token séparément
- Deux **connexions résiduelles** par bloc : l'idée de ResNet. Le bloc est répété **12 fois** dans DeiT-tiny

Source : Dosovitskiy et al., ICLR 2021, figure 1 et équations 2 à 4.

Biais inductifs (a priori) et volume de données

Pourquoi : avec 30 images par race, c'est ce point qui peut désavantager le ViT.

Ce qu'un CNN sait d'avance

- **Localité** : les pixels voisins sont liés
- **Même réponse partout** : un motif est reconnu où qu'il soit dans l'image

Le ViT

Beaucoup moins d'a priori : seuls le découpage en patches et le MLP appliqué à chaque token sont locaux. Le reste s'apprend à partir des données.

ViT entraîné sur ImageNet seul

1,28 M d'images

quelques points sous un ResNet de taille comparable

ViT pré-entraîné à grande échelle

14 M à 300 M d'images

approche ou dépasse les meilleurs CNN

Conséquence pour le cas d'étude : ne pas entraîner un ViT de zéro sur peu d'images, partir d'un ViT pré-entraîné.

Dosovitskiy et al., An Image is Worth 16x16 Words, ICLR 2021 : « large scale training trumps inductive bias ».

DeiT : entraîner un ViT sur ImageNet seul

Pourquoi : DeiT-tiny est le ViT que l'on compare au TD.

DeiT obtient un ViT compétitif sans pré-entraînement massif, grâce à une recette d'entraînement soignée :

- **Augmentation de données forte** et régularisation
- **Distillation** : le ViT apprend aussi à imiter un CNN enseignant, grâce à un token supplémentaire (le DeiT-tiny du TD est la version sans distillation)
- DeiT-tiny et DeiT-small : **trois jours sur une machine à 4 GPU** (DeiT-B : deux à trois jours sur 8 GPU)

5,7 M

paramètres pour DeiT-tiny (recomptés)

72,2 %

accuracy top-1 sur ImageNet (sans distillation), contre 77,7 % pour EfficientNet-B0 à taille comparable

Pour le cas d'étude : **DeiT-tiny**, troisième candidat. Il demande 1,25 milliard d'opérations par image, contre 0,39 pour EfficientNet-B0 : les temps se mesureront au TD.

Touvron et al., ICML 2021. Accuracy : dépôt officiel DeiT. Opérations : multiplications-additions, comptées avec torch.utils.flop_counter, attention comprise.

Le Vision Transformer : à retenir

1. L'**attention** : chaque élément fait la moyenne des valeurs de tous les éléments, pondérée par la ressemblance entre sa **requête** et leurs **clés**.

2. Le ViT découpe l'image en **patches** 16 × 16, qui deviennent des tokens ; avec un token **[CLS]** et des positions, ils traversent un encodeur Transformer, et la tête lit [CLS].

3. Il voit toute l'image dès la première couche, mais a **moins d'a priori** qu'un CNN : il lui faut un pré-entraînement massif, ou la recette de DeiT, pour rivaliser.

Pour approfondir : d2l, section 11.3 : Attention Scoring Functions ; d2l, section 11.8 : Transformers for Vision

TD · étapes 2 et 4

Retour au cas d'étude : trois candidats

Pourquoi : répondre à la première question : quels modèles comparer, et comment.

Candidat	Famille	Paramètres	Accuracy top-1 (ImageNet)	Dans le TD
ResNet-18	CNN résiduel	11,7 M	69,8 %	étapes 2 et 4
EfficientNet-B0	CNN, compound scaling	5,3 M	77,7 %	étapes 2 et 4 ; fine-tuning imposé (étape 6)
DeiT-tiny	Vision Transformer	5,7 M	72,2 %	étapes 2 et 4

Le meilleur sur ImageNet n'est pas forcément le meilleur sur nos 37 races : on compare leurs features sur la **validation**, à l'étape 4.

Fine-tuning sur EfficientNet-B0 : le moins d'opérations par image (0,39 milliard, contre 1,8 et 1,25), à confirmer par les temps mesurés à l'étape 2. Sources : torchvision, dépôt DeiT.



Après la partie 4

Pause

Reprise avec la méthode : comparer proprement, puis adapter les modèles. Pendant la pause, lancer uv sync puis uv run python verifier_installation.py --donnees (environ 1 Go à télécharger)



Partie 5

Comparer proprement : validation, test et baseline

Pourquoi : la règle qui rend fiables toutes les comparaisons du TD, dès l'étape 3

Comparer des modèles : définition

Définition

Comparer des modèles, c'est mesurer leur accuracy sur des images qu'ils n'ont pas vues, avec une règle fixe : on choisit sur la validation, on mesure une seule fois sur le test, et on compare toujours à une méthode simple, la baseline.

Le problème que cela résout

Sans cette règle, le score annoncé est trop optimiste, et l'on ne sait pas si le deep learning vaut son coût.

Ce qui entre, ce qui sort

Entrée : des modèles entraînés et trois jeux d'images distincts. Sortie : un modèle choisi et un score fiable.

Ce que l'on va voir, dans l'ordre

2. Validation et test

3. L'incertitude d'une mesure

4. Baseline de machine learning classique

5. HOG : décrire les contours à la main

6. À retenir

Dans le TD : Étapes 3, 4, 5 et 7.

Validation et test

Pourquoi : c'est le découpage de l'étape 3, et la seule façon de choisir sans tricher.

Dans le TD : 30 / 10 / 15 images par race, 37 races. Validation : étapes 4 à 6. Test : étape 7, une seule fois.

Entraînement	Validation	Test
apprendre les poids	comparer et choisir : méthodes, réglages	mesurer, une seule fois, à la fin

Choisir une configuration en regardant le test est une **fuite de données** : le score annoncé devient optimiste, et ne dit plus rien des images nouvelles.

Pourquoi trois jeux : chaque choix fait en regardant un jeu adapte un peu le modèle à ce jeu. Seul un jeu jamais regardé pendant les réglages mesure honnêtement ce que le modèle vaut sur des images nouvelles.

Source : d2l, section 3.6.3 (sélection de modèle).

L'incertitude d'une mesure

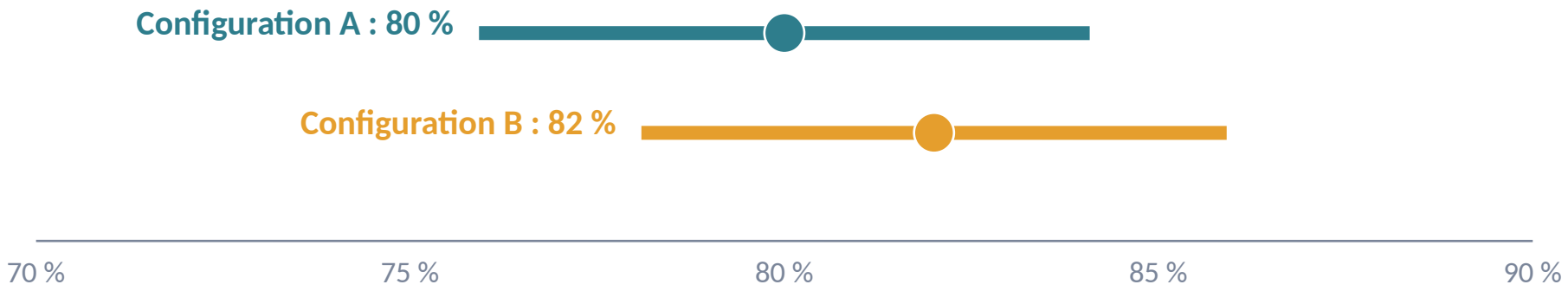
Pourquoi : savoir quand un écart entre deux modèles n'est que du bruit, avant de comparer vos chiffres.

Une accuracy mesurée sur quelques centaines d'images n'est connue qu'à quelques points près : un petit écart entre deux modèles peut n'être que du **bruit**.

± 4 points

pour une accuracy de 80 % mesurée sur nos 370 images de validation

Calcul, pour les curieux : $p \pm 1{,}96 \cdot \sqrt{p\,(1-p)\,/\,n}$, avec p l'accuracy mesurée et n le nombre d'images (intervalle de Wald, à 95 %) : $1{,}96 \times \sqrt{0{,}8 \times 0{,}2\,/\,370} \approx 0{,}04$.

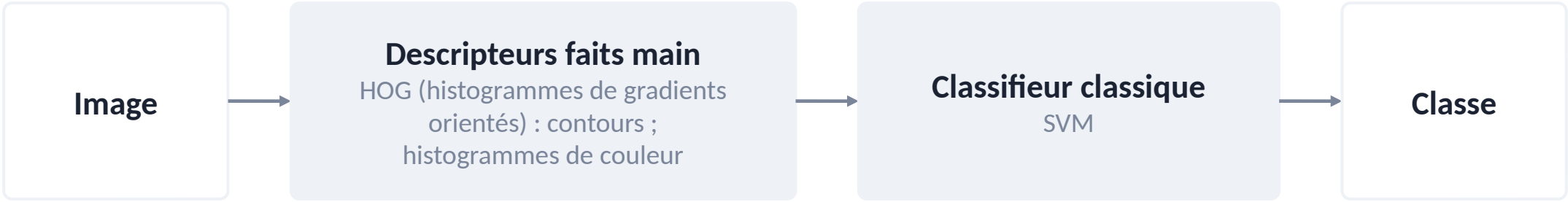


Les intervalles se recouvrent : 2 points ne départagent pas A et B.

Source : Intervalle de Wald pour une proportion ; d2l, section 3.6.3.

Baseline de machine learning classique

Pourquoi : c'est l'étape 5 du TD : la référence sans deep learning.



- **C du SVM** : compromis entre une marge large et peu d'erreurs d'entraînement ; choisi sur la validation, comme pour les modèles profonds
- **Pourquoi** une baseline : savoir si le deep learning vaut son coût (calcul, données, maintenance)
- Rapide et interprétable, elle suffit parfois ; le deep learning gagne quand les classes se distinguent par des détails que personne ne saurait décrire à la main

Au TD (étape 5), on mesure sur la validation l'écart entre les modèles profonds et cette baseline, puis on juge s'il justifie le coût supplémentaire.

Source : scikit-learn, SVC ; scikit-image, fonction hog.

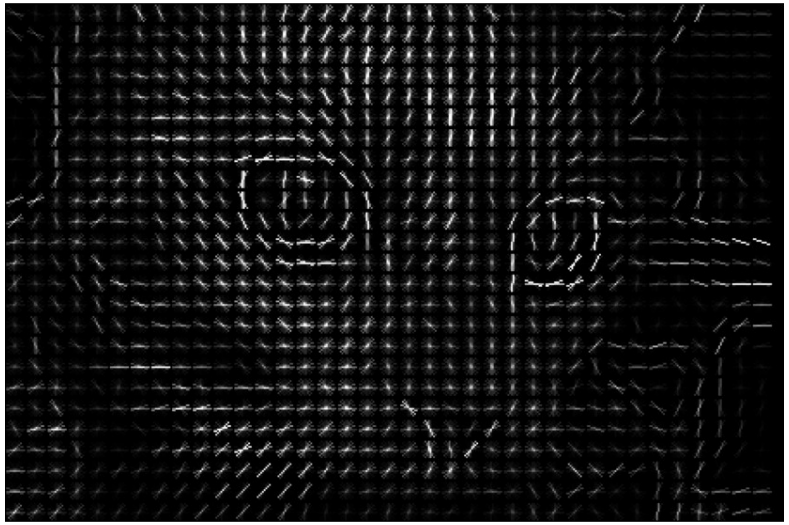
HOG : décrire les contours à la main

Pourquoi : c'est le descripteur de la baseline, que vous calculerez au TODO 2.

Image d'entrée (niveaux de gris)



HOG : orientations du gradient par cellule



- Le **gradient** donne, en chaque pixel, la direction où l'intensité change
- Dans chaque **cellule**, un histogramme des orientations du gradient
- Une **normalisation** par blocs de cellules voisines, pour moins dépendre de l'éclairage

HOG calculé avec scikit-image (cellules de 12 pixels ; le TD utilise d'autres réglages).

Les **histogrammes de couleur** complètent HOG : ils décrivent la distribution des couleurs, sans information spatiale.

Source : Dalal et Triggs, Histograms of Oriented Gradients, CVPR 2005 ; scikit-image, fonction hog.

Comparer des modèles : à retenir

1. Trois jeux distincts : l'**entraînement** apprend, la **validation** sert à choisir, le **test** mesure une seule fois, à la fin.

2. Sur quelques centaines d'images, un écart de **1 ou 2 points** entre deux modèles peut n'être que du bruit.

3. Toujours comparer à une **baseline** classique, réglée elle aussi sur la validation : elle dit si le deep learning vaut son coût.

Pour approfondir : d2l, section 3.6 : Generalization (sélection de modèle)



Partie 6

Transfer learning et feature extraction

Pourquoi : réutiliser un réseau pré-entraîné sans le réentraîner, c'est l'étape 4 du TD

Le transfer learning : définition

Définition

Le transfer learning consiste à réutiliser un réseau déjà entraîné sur une grande base d'images, ImageNet, pour une nouvelle tâche, au lieu d'entraîner un réseau de zéro.

Le problème que cela résout

Entraîner de zéro demande des millions d'images annotées et des jours de calcul. Nous avons 30 images par classe et pas de GPU.

Ce qui entre, ce qui sort

Entrée : un réseau pré-entraîné et quelques images annotées. Sortie : un réseau adapté à nos classes.

Ce que l'on va voir, dans l'ordre

2. Ce que l'on réutilise d'un réseau pré-entraîné

3. Trois stratégies d'adaptation

4. Choix de la stratégie

5. Feature extraction en PyTorch

6. À retenir

Dans le TD : Tout l'après-midi : étapes 4 à 8.

Ce que l'on réutilise d'un réseau pré-entraîné

Pourquoi : c'est le principe du transfer learning, utilisé tout l'après-midi.



1,28 M

images d'entraînement dans ImageNet-1k, en 1000 classes : c'est cette variété qui rend les premières couches réutilisables

Les premières couches apprennent des motifs utiles à presque toutes les images ; les dernières sont propres aux classes d'ImageNet. Le transfert marche d'autant mieux que la nouvelle tâche ressemble à l'ancienne.

Yosinski et al., How transferable are features in deep neural networks?, NIPS 2014 (aujourd'hui NeurIPS).

Trois stratégies d'adaptation

Pourquoi : le TD compare les deux stratégies de transfert, aux étapes 4 et 6 ; l'entraînement de zéro sert de point de comparaison.

Entraîner de zéro



sans transfert : tout est appris depuis des poids aléatoires

Quand : référence de comparaison ; même loin d'ImageNet, partir des poids pré-entraînés aide souvent

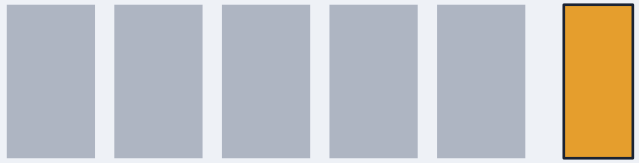
Fine-tuning



la tête et les derniers étages (ou tout le réseau) sont réentraînés

Quand : plus de données ; d'autant plus d'étages dégelés que le domaine est éloigné d'ImageNet

Feature extraction



seule la nouvelle tête est entraînée, sur les features du backbone

Quand : peu de données, domaine proche

gelé entraîné

Étage : un groupe de couches consécutives du backbone (layer1 à layer4 pour ResNet-18, les 9 éléments de features pour EfficientNet-B0).

Source : CS231n, Transfer Learning ; tutoriel PyTorch, Transfer Learning for Computer Vision.

Choix de la stratégie

Pourquoi : savoir par quelle stratégie commencer, selon ses données.

	Peu de données	Beaucoup de données
Proche d'ImageNet	**Feature extraction** entraîner seulement une nouvelle tête suffit souvent **Oxford Pets (TD 1) : 30 images par race ; ImageNet contient 118 races de chiens**	**Fine-tuning** des derniers étages, voire de tout le réseau
Éloigné d'ImageNet	**Le cas difficile** prendre les features de couches plus proches de l'entrée, moins propres à ImageNet ; ou dégeler peu d'étages et augmenter les données **NEU-DET (projet) : 30 images par défaut, acier en niveaux de gris**	**Fine-tuning profond** dégeler tout le réseau, en partant des poids pré-entraînés

Cette grille donne un point de départ ; on tranche en mesurant sur la validation.

Source : CS231n, Transfer Learning (les quatre cas selon la taille des données et la distance au jeu d'origine).

Feature extraction en PyTorch

Pourquoi : la première stratégie, la moins coûteuse : c'est l'étape 4 du TD (TODO 1).

```
import torch
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader
from torchvision.models import resnet18, ResNet18_Weights

modele = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1).eval()
modele.fc = torch.nn.Identity()  # sans tête : il renvoie ses features
def embeddings(jeu):         # un vecteur de features par image
    vecteurs, classes = [], []
    with torch.no_grad():
        for x, y in DataLoader(jeu, batch_size=64):   # sans mélange
            vecteurs.append(modele(x))                # (lot, 512)
            classes.append(y)
    return torch.cat(vecteurs).numpy(), torch.cat(classes).numpy()

# ici, deux jeux sans augmentation : aucun tirage aléatoire
e_tr, y_tr = embeddings(jeu_entrainement)   # calculés une seule fois
e_va, y_va = embeddings(jeu_validation)
print(e_tr.shape)           # (1110, 512) : 37 races x 30 images
classifieur = make_pipeline(StandardScaler(),
                            LogisticRegression(max_iter=3000))
accuracy = classifieur.fit(e_tr, y_tr).score(e_va, y_va)  # étape 4 du TD
```

- **nn.Identity()** à la place de la tête : le modèle renvoie un vecteur de 512 features par image, que le TD appelle **embedding** de l'image
- Calculés **une seule fois**, sans gradient : le backbone ne change pas
- Une **régression logistique** les classe : c'est la nouvelle tête
- Dans le TD (étape 4) : **tp.extraire_embeddings** fait la boucle

Source : Tutoriel PyTorch, Transfer Learning for Computer Vision : ConvNet as fixed feature extractor.

Le transfer learning : à retenir

1. Un réseau pré-entraîné sur ImageNet a appris des features **réutilisables**, surtout dans ses premières couches.

2. Deux stratégies de transfert, comparées à l'**entraînement de zéro** : la **feature extraction** (tête remplacée par **nn.Identity()**, features calculées une fois, classifieur simple dessus) et le **fine-tuning**.

3. Le choix dépend de la **quantité de données** et de la **distance à ImageNet** ; il se tranche en mesurant sur la validation.

Pour approfondir : CS231n : Transfer Learning ;
Tutoriel PyTorch : Transfer Learning for Computer Vision (dont ConvNet as fixed feature extractor)



Partie 7

Fine-tuning

Pourquoi : réentraîner les derniers étages, c'est l'étape 6, la plus longue du TD

Le fine-tuning : définition

Définition

Le fine-tuning poursuit l'entraînement des poids pré-entraînés, de tout le réseau ou de ses derniers étages, en même temps que la nouvelle tête, avec un petit learning rate.

Le problème que cela résout

Quand les images diffèrent de celles d'ImageNet, les features figées ne suffisent pas toujours : le fine-tuning les ajuste à la nouvelle tâche.

Ce qui entre, ce qui sort

Entrée : un réseau pré-entraîné et des images annotées. Sortie : un réseau dont les poids sont ajustés à la nouvelle tâche ; avec peu d'images, le TD n'ajuste que les derniers étages.

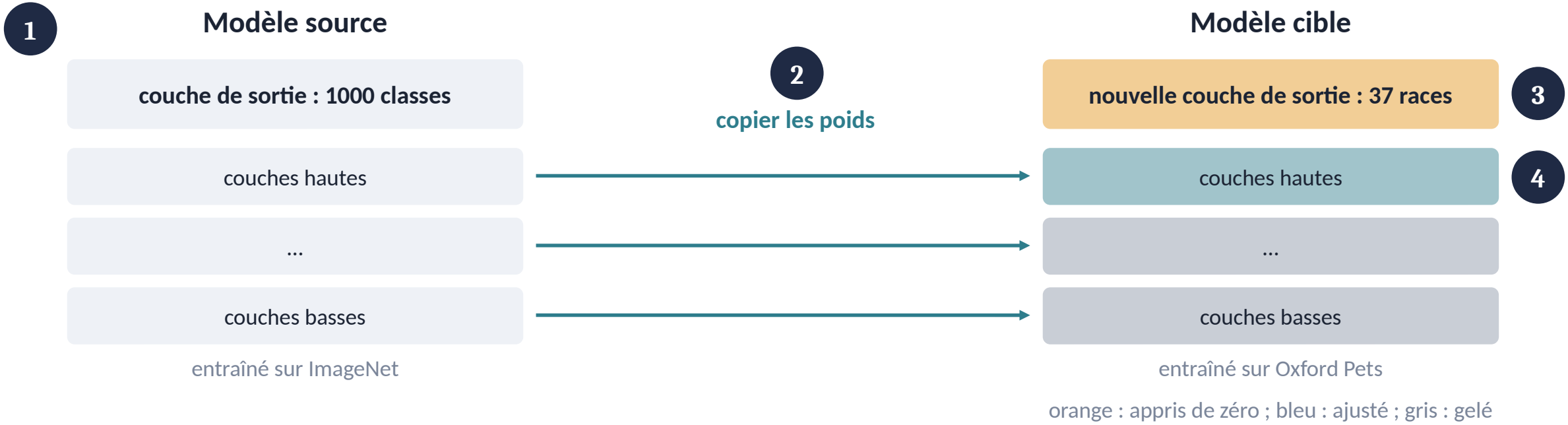
Ce que l'on va voir, dans l'ordre

2. Procédure de fine-tuning
3. Fine-tuning : les six étapes
4. Geler et dégeler des couches
5. Learning rates différenciés
6. Groupes de paramètres dans l'optimiseur
7. Prétraitement et augmentation
8. Boucle d'entraînement et mode évaluation
9. Erreurs fréquentes en fine-tuning
10. À retenir

Dans le TD : Étape 6 (TODO 3, 4 et 5).

Procédure de fine-tuning

Pourquoi : ce que l'on garde du modèle pré-entraîné, et ce que l'on change.



Avec 30 images par race, le TD n'ajuste que les **derniers étages** ; d2l ajuste toutes les couches copiées, avec un learning rate dix fois plus faible.

Figure d'après Dive into Deep Learning, section 14.2 (Fine-Tuning).

Fine-tuning : les six étapes

Pourquoi : c'est le déroulé de l'étape 6 du TD, dans l'ordre.

1 Source

un réseau pré-entraîné sur ImageNet : on télécharge ses poids

2 Copier

toutes les couches et leurs poids, sauf la couche de sortie, propre aux 1000 classes d'ImageNet

3 Nouvelle tête

une sortie par classe (37 races), initialisée au hasard

4 Entraîner

la tête de zéro ; les derniers étages ajustés avec un learning rate plus faible ; les autres gelés

5 Comparer

les configurations sur la validation : étages dégelés, learning rate, époques

6 Mesurer

une seule fois sur le test, à la fin, avec la configuration retenue

Source : Étapes 1 à 4 : Dive into Deep Learning, section 14.2 ; étapes 5 et 6 : la règle du module.

Geler et dégeler des couches

Pourquoi : c'est le TODO 3 du TD, montré ici sur ResNet-18.

```python
import torch.nn as nn
from torchvision.models import resnet18, ResNet18_Weights

modele = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
modele.fc = nn.Linear(modele.fc.in_features, 37)  # nouvelle tête, 37 races

for p in modele.parameters():          # feature extraction : tout gelé...
    p.requires_grad = False
for p in modele.fc.parameters():       # ...sauf la nouvelle tête
    p.requires_grad = True

for p in modele.layer4.parameters():   # fine-tuning : dégeler layer4
    p.requires_grad = True

entrainables = sum(p.numel() for p in modele.parameters()
                   if p.requires_grad)
print(entrainables)   # 8412709 : 18981 pour la tête, 8393728 pour layer4
```

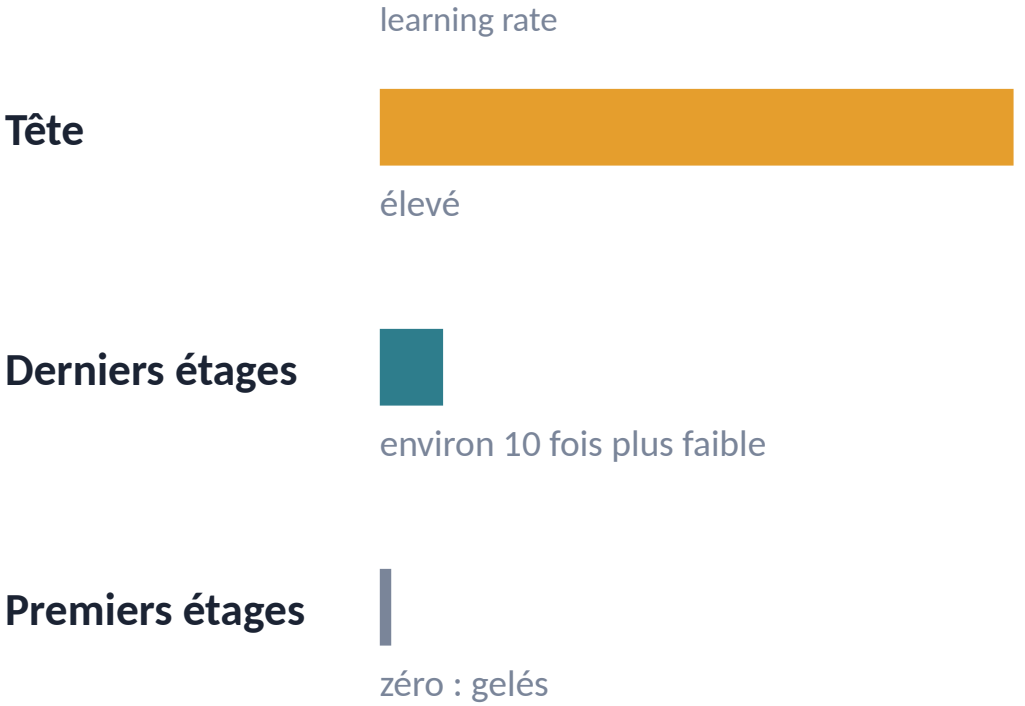
- **requires_grad = False** : le paramètre ne reçoit plus de gradient
- Ordre : remplacer la tête, tout geler, puis dégeler ce que l'on entraîne
- **Compter les paramètres entraînés** : le contrôle le plus simple
- Un **étage** : un élément de features (9 pour EfficientNet-B0), ou layer1 à layer4 pour ResNet-18 ; dégeler n étages, c'est dégeler les n derniers

Source : Tutoriel PyTorch, Transfer Learning for Computer Vision (requires_grad).

Learning rates différenciés

Pourquoi : sans eux, le fine-tuning peut déformer trop vite ce que le réseau a appris sur ImageNet.

- La nouvelle tête part de **poids aléatoires** : ses gros gradients remontent dans le backbone
- Avec un learning rate élevé partout, ils peuvent **déformer trop vite** les poids pré-entraînés et détruire des features utiles
- Solution : des **groupes de paramètres** dans l'optimiseur, chacun avec son learning rate



Un facteur 10 entre la tête et le backbone : un **point de départ courant**, à régler sur la validation.

Source : d2l, section 14.2 (Fine-Tuning) ; CS231n, Transfer Learning.

Groupes de paramètres dans l'optimiseur

Pourquoi : c'est le TODO 4 du TD.

```python
import torch

optimiseur = torch.optim.AdamW([
    {"params": modele.fc.parameters(),     "lr": 1e-3},   # tête : de zéro
    {"params": modele.layer4.parameters(), "lr": 1e-4},   # dernier étage
], weight_decay=1e-4)

for groupe in optimiseur.param_groups:
    print(groupe["lr"], sum(p.numel() for p in groupe["params"]))
# 0.001 18981
# 0.0001 8393728
```

- Un **groupe** : une liste de paramètres et ses propres réglages
- Le learning rate de la tête vaut 10 fois celui du dernier étage
- Les paramètres gelés ne sont pas donnés à l'optimiseur
- **weight_decay** s'applique aux deux groupes

Source : Documentation PyTorch : torch.optim, options par groupe de paramètres.

Prétraitement et augmentation

Pourquoi : un prétraitement différent de celui du pré-entraînement suffit à faire échouer le transfert.

```
from torch.utils.data import DataLoader
from torchvision import transforms as T

# moyenne et écart-type d'ImageNet, canal par canal
moyenne, ecart_type = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]

pour_entrainement = T.Compose([
    T.RandomResizedCrop(160, scale=(0.7, 1.0)),   # recadrage aléatoire
    T.RandomHorizontalFlip(),                     # retournement aléatoire
    T.ToTensor(),
    T.Normalize(moyenne, ecart_type),
])
pour_evaluation = T.Compose([                     # rien d'aléatoire
    T.Resize(182), T.CenterCrop(160),             # le cadrage du TD
    T.ToTensor(), T.Normalize(moyenne, ecart_type),
])

chargeur_entrainement = DataLoader(jeu_entrainement, batch_size=32,
                                   shuffle=True)
chargeur_validation = DataLoader(jeu_validation, batch_size=64)
```

- **Normalize** avec les statistiques d'ImageNet, celles du pré-entraînement
- Transformations aléatoires **pour l'entraînement seulement**
- **Resize(182) puis CenterCrop(160)** : le cadrage du TD, pour la validation et le test (160 × 256 / 224 = 182,9, dont on garde la partie entière : le rapport 256 / 224 d'ImageNet)
- **jeu_entrainement**, **jeu_validation** : des Dataset qui appliquent ces transformations ; **shuffle=True** pour l'entraînement seulement

Source : Documentation torchvision : transforms ; d2l, section 14.1.

Boucle d'entraînement et mode évaluation

Pourquoi : c'est le TODO 5 du TD ; oublier eval() est l'erreur la plus fréquente.

```
critere = nn.CrossEntropyLoss()

for epoque in range(3):
    modele.train()                # BatchNorm, dropout : mode entraînement
    for x, y in chargeur_entrainement:
        optimiseur.zero_grad()
        perte = critere(modele(x), y)
        perte.backward()
        optimiseur.step()

    modele.eval()                 # statistiques mémorisées, sans dropout
    bons = 0
    with torch.no_grad():         # pas de gradient à l'évaluation
        for x, y in chargeur_validation:
            bons += (modele(x).argmax(dim=1) == y).sum().item()
    accuracy = bons / len(chargeur_validation.dataset)
    print(f"époque {epoque} : accuracy de validation {accuracy:.1%}")
```

- **modele.train()** avant d'entraîner, **modele.eval()** avant d'évaluer
- **zero_grad, backward, step** : dans cet ordre, à chaque lot
- **torch.no_grad()** : évaluation plus rapide, sans mémoire pour les gradients
- Suite des diapositives précédentes : modele, optimiseur, chargeurs

Source : Tutoriel PyTorch, Learn the Basics : Optimizing Model Parameters.

Erreurs fréquentes en fine-tuning

Pourquoi : trois pièges du TD que le code ne montre pas.

BatchNorm

En mode entraînement, elle normalise avec les statistiques du lot et met à jour sa moyenne et sa variance mémorisées, même si ses poids sont gelés. Pour prédire : eval(). D'où un écart entre la tête seule (étape 6) et la feature extraction (étape 4).

Résolution

Le TD travaille en 160 px au lieu de 224, pour tenir sur CPU, au prix d'un peu d'accuracy : la référence « 0 étage dégelé » est donc mesurée elle aussi en 160 px.

Surapprentissage

Avec 30 images par classe, il arrive vite : suivre la perte et l'accuracy de validation à chaque époque.

Source : Documentation PyTorch, BatchNorm2d.

Le fine-tuning : à retenir

1. On garde le réseau pré-entraîné sauf sa tête, on ajoute une **nouvelle tête**, puis on poursuit l'entraînement de tout ou partie du réseau ; dans le TD, la tête et les **derniers étages**, les autres restant gelés.

2. Un **learning rate plus faible** pour les étages pré-entraînés que pour la tête, qui part de poids aléatoires.

3. Même **prétraitement** qu'au pré-entraînement, **augmentation** sur l'entraînement seulement, **model.eval()** pour évaluer.

Pour approfondir : d2l, section 14.2 : Fine-Tuning ; Tutoriel PyTorch : Finetuning the ConvNet

Partie 8

Évaluation et diagnostic

Vérifier le modèle retenu et comprendre ses erreurs avant de le livrer : les étapes 7 et 8

Diagnostiquer un modèle : définition

Définition

Diagnostiquer un modèle, c'est regarder au-delà de son score : la matrice de confusion montre quelles classes il confond, et Grad-CAM, une carte de chaleur, montre sur quelles zones de l'image il s'appuie pour une classe choisie.

Le problème que cela résout

Un bon score peut cacher des confusions entre classes, une classe rare ratée, ou un modèle qui s'appuie sur un mauvais indice, comme le fond de l'image.

Ce qui entre, ce qui sort

Entrée : les prédictions, puis une image et la classe visée. Sortie : des scores, un tableau des confusions, une carte de chaleur.

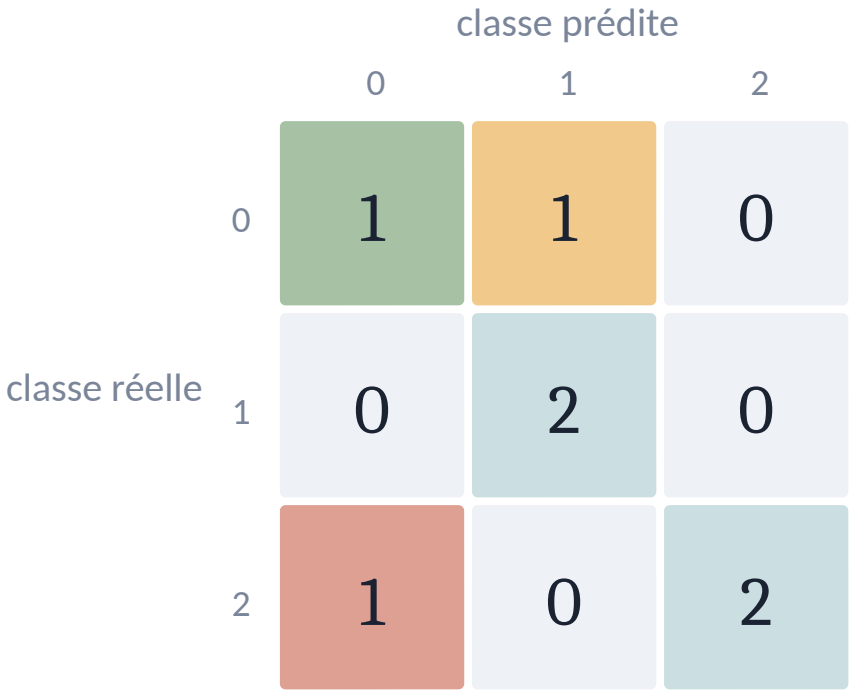
Ce que l'on va voir, dans l'ordre

2. La matrice de confusion
3. Précision, rappel et F1
4. Grad-CAM : les zones de l'image qui comptent
5. Grad-CAM sur un exemple jouet
6. Lire les activations avec un crochet
7. Crochet sur un tenseur : lire un gradient
8. Exemple : un biais lié à la source des images
9. À retenir

Dans le TD : Étapes 7 (TODO 6) et 8 (TODO 7).

La matrice de confusion

Pourquoi : l'accuracy seule ne dit pas où le modèle se trompe : c'est l'étape 7 du TD.



7 images, 3 classes

- vrai positif (VP) de la classe 0
- faux négatif (FN) : une image de 0 prédite 1
- faux positif (FP) : une image de 2 prédite 0
- diagonale : les bonnes réponses

Accuracy : 1 + 2 + 2 = 5 bonnes réponses sur 7, soit **0,71**

Les cases hors diagonale les plus remplies désignent les **paires de classes confondues** : ce sont elles qu'on examine ensuite, avec Grad-CAM.

Source : scikit-learn, section 3.4 : confusion_matrix ; le code est en annexe.

Précision, rappel et F1

Pourquoi : les métriques qui comptent chaque classe, même rare.

- **Précision** d'une classe : VP / (VP + FP), la part de justes parmi les images prédites dans cette classe
- **Rappel** : VP / (VP + FN), la part retrouvée parmi les images de cette classe
- **F1** : 2 · précision · rappel / (précision + rappel)
- **F1 macro** : la moyenne des F1 des classes

Classe	Précision	Rappel	F1
0	1 / 2 = 0,5	1 / 2 = 0,5	0,5
1	2 / 3 ≈ 0,67	2 / 2 = 1	0,8
2	2 / 2 = 1	2 / 3 ≈ 0,67	0,8

F1 macro = (0,5 + 0,8 + 0,8) / 3 = **0,7** ; accuracy = 5 / 7 ≈ 0,71



Le **F1 macro** donne le même poids à chaque classe, même rare : sur des classes déséquilibrées, il révèle ce que l'accuracy cache.

Source : scikit-learn, section 3.4 (Metrics and scoring).

Grad-CAM : les zones de l'image qui comptent

Pourquoi : vérifier sur quoi le modèle s'appuie, au-delà de son score : étape 8 du TD.

Grad-CAM colorie les zones de l'image qui ont le plus poussé le modèle vers sa réponse. On vérifie ainsi qu'il regarde l'animal, et pas le fond.

1

Faire passer l'image

et garder les cartes de la dernière couche de convolution

→

2

Calculer les gradients

comment le score de la classe change quand chaque carte change

→

3

Donner un poids à chaque carte

la moyenne de ses gradients : une carte utile a un grand poids

→

4

Additionner les cartes

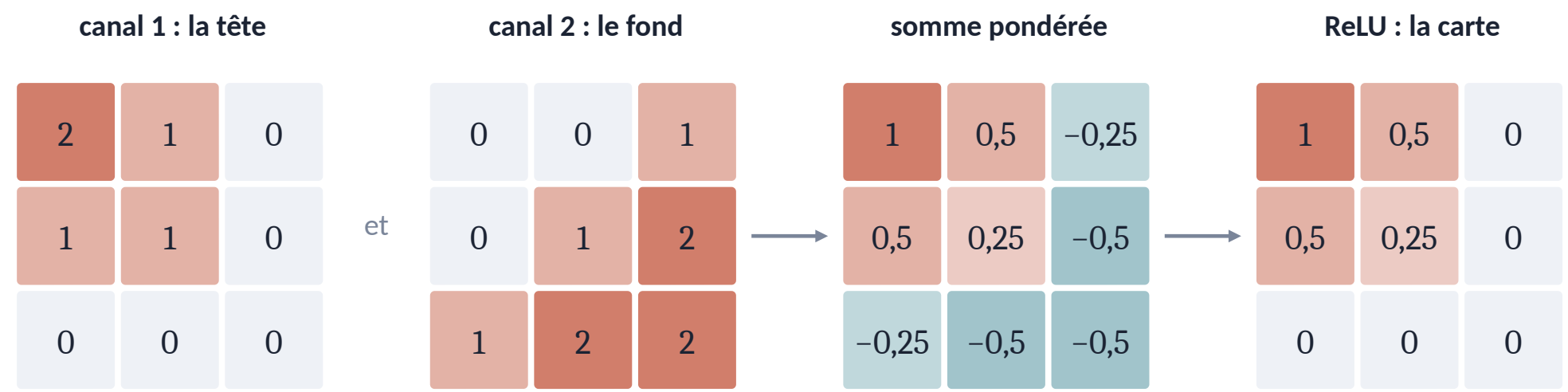
avec ces poids, garder le positif : c'est la carte de chaleur

La carte est **grossière** (5 × 5 au TD) : elle montre une zone, pas un contour. Elle aide à **repérer** un problème ; elle ne prouve pas que le modèle est bon.

Selvaraju et al., Grad-CAM, ICCV 2017 : la formule est l'équation 1 (poids) et 2 (carte) de l'article.

Grad-CAM sur un exemple jouet

Pourquoi : refaire le calcul de Grad-CAM à la main, sur deux petites cartes.



$\alpha_1 = 0{,}5$: il fait monter le score

$\alpha_2 = -0{,}25$: il le fait baisser

0,5 × canal 1 − 0,25 × canal 2

le positif seulement

Les α sont les moyennes des gradients, données par la passe arrière. Le canal de la tête l'emporte : la carte, agrandie à la taille de l'image, **montre la tête**.

Exemple jouet calculé avec PyTorch.

Lire les activations avec un crochet

Pourquoi : pour coder Grad-CAM, il faut d'abord lire les activations d'une couche interne.

```
activations = {}

def garder(module, entree, sortie):     # appelée à chaque passe avant
    activations["layer4"] = sortie.detach()

crochet = modele.layer4.register_forward_hook(garder)
modele.eval()
with torch.no_grad():
    modele(torch.randn(1, 3, 224, 224))
crochet.remove()                        # toujours retirer un crochet après usage

print(activations["layer4"].shape)      # torch.Size([1, 512, 7, 7])
```

- Grad-CAM a besoin des activations d'une couche interne, puis de leurs gradients : PyTorch les donne par des **crochets**
- Un **crochet** (hook) appelle une fonction à chaque passe avant
- On récupère la sortie d'une couche sans modifier le modèle
- **crochet.remove()** : sinon il reste actif
- Ici, **no_grad()** et **.detach()** : on ne veut que les activations. Grad-CAM a aussi besoin des gradients (diapositive suivante)

Source : Documentation PyTorch : torch.nn.Module.register_forward_hook.

Crochet sur un tenseur : lire un gradient

Pourquoi : Grad-CAM a aussi besoin des gradients de ces activations (TODO 7).

```python
import torch

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x ** 2              # un tenseur intermédiaire, comme une activation
gradients = {}

def garder(g):          # appelée pendant la rétropropagation
    gradients["y"] = g

y.register_hook(garder) # crochet sur un tenseur, pas sur un module
z = (3 * y).sum()
z.backward()
print(gradients["y"])   # tensor([3., 3., 3.]) : dz/dy
print(x.grad)           # tensor([ 6., 12., 18.]) : dz/dx = 3 * 2x
```

- **register_hook** sur un **tenseur** : la fonction reçoit le gradient, par rapport à ce tenseur, de la quantité rétropropagée (ici z ; pour Grad-CAM, le score de la classe)
- Elle est appelée pendant **backward()**
- Pour Grad-CAM (TODO 7) : dans le crochet de passe avant, garder **sortie** et appeler **sortie.register_hook**
- Ni **no_grad()** ni **.detach()** avant la rétropropagation : sinon, aucun gradient

Source : Documentation PyTorch : torch.Tensor.register_hook.

Exemple : un biais lié à la source des images

Pourquoi : un bon score peut cacher un modèle qui s'appuie sur le mauvais indice.

Radiographies et pneumonie

- Des CNN entraînés à reconnaître **l'hôpital d'origine** d'une radiographie y parvenaient dans 99,95 % (NIH) et 99,98 % (Mount Sinai) des cas : l'image porte la trace de sa source
- Dans **3 des 5 comparaisons** de l'étude, les CNN de détection de pneumonie étaient significativement moins performants sur les radiographies d'un autre hôpital : le score mesuré sur les hôpitaux d'entraînement surestimait la performance réelle

Zech et al., PLOS Medicine, 2018

Indices parasites fréquents

- le fond de l'image
- un filigrane, un texte incrusté
- le cadrage
- l'appareil ou la source des images

Matrice de confusion et Grad-CAM aident à **repérer** ce type de problème ; ils ne garantissent pas que le modèle est fiable.

Source : Zech et al., Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs, PLOS Medicine 2018.

Diagnostiquer un modèle : à retenir

1. L'**accuracy** suffit quand les classes sont équilibrées ; le **F1 macro** donne le même poids à chaque classe, même rare.

2. La **matrice de confusion** (lignes : classe réelle, colonnes : classe prédite) montre quelles classes sont confondues.

3. **Grad-CAM** colorie les zones de l'image qui ont compté pour la réponse : il aide à **repérer** un indice parasite, sans prouver que le modèle est fiable.

Pour approfondir : scikit-learn, section 3.4 : Metrics and scoring ; Selvaraju et al., 2017 : l'article Grad-CAM (arXiv)

TD · étape 10

Synthèse

Pourquoi : les réponses aux trois questions, et le coût, à garder en tête pour le TD.

1. **Quel modèle ?** Trois candidats pré-entraînés : ResNet-18 (connexions résiduelles), EfficientNet-B0 (compound scaling) et DeiT-tiny (Vision Transformer, qui demande plus de données qu'un CNN). On choisit sur la validation, pas d'après le classement sur ImageNet.

2. **Quel coût ?** Nombre de paramètres, nombre d'opérations et temps mesuré sur la machine cible sont trois grandeurs différentes : seul le temps mesuré dit si le modèle convient à la machine.

3. **Comment l'adapter ?** Feature extraction ou fine-tuning, selon la quantité de données et la distance au domaine d'origine ; un learning rate plus faible pour le backbone.

4. **Comment vérifier qu'il fonctionne ?** Régler sur la validation, mesurer une seule fois sur le test, face à une baseline classique ; puis la matrice de confusion et Grad-CAM pour comprendre les erreurs.

Déroulé du TD 1

Pourquoi : chaque étape du TD applique une partie du cours.

Étape 2

Charger les trois modèles, prédire sur des photos, mesurer leur temps

parties 2 à 4

Étape 3

Le jeu Oxford Pets : 30 / 10 / 15 images par race

partie 5

Étape 4

Feature extraction : trois candidats, sur la validation

partie 6

Étape 5

Baseline HOG, couleur et SVM, sur la validation

partie 5

Étape 6

Fine-tuning d'EfficientNet-B0, sur la validation

partie 7

Étape 7

Mesure finale sur le test, une seule fois ; matrice de confusion

partie 8

Étape 8

Grad-CAM sur les réussites et les erreurs

partie 8

Étape 9

Mini-application : votre propre photo

partie 8

Étape 10

Fiche de décision : le modèle retenu et sa justification chiffrée

synthèse

Étape 11

Brique 1 du projet : les défauts d'acier, sur une validation

parties 5 à 7



Étapes 0 et 1 : vérifier l'installation (si ce n'est pas fait à la pause : **uv sync**, puis **uv run python verifier_installation.py --donnees**) et revoir le cours en bref.

Pendant le TD : sur les diapositives du cours, la pastille orange « TD · étape N » en haut à gauche indique l'étape, et le TODO, où elles servent.



Annexe

Le code et les détails

Non présentées en cours, à lire seul : le code PyTorch des briques, le détail du bloc MBConv et le code des métriques, dans l'ordre du cours

Convolution en PyTorch

Pourquoi : écrire une convolution et compter ses paramètres, comme dans le TD.

```python
import torch
import torch.nn as nn

conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
x = torch.randn(8, 3, 224, 224)    # un lot de 8 images RGB : (N, C, H, W)

y = conv(x)
print(y.shape)              # torch.Size([8, 64, 224, 224])
print(conv.weight.shape)    # torch.Size([64, 3, 3, 3])
print(sum(p.numel() for p in conv.parameters()))   # 1792 = 1728 + 64 biais
```

- Un lot d'images a la forme **(N, C, H, W)** : lot, canaux, hauteur, largeur
- **padding=1** conserve la taille 224 × 224 avec un noyau 3 × 3
- Les poids : **64 filtres de 3 × 3 × 3**, plus un biais par filtre : les 1 792 de « Taille de la sortie et nombre de poids »
- **p.numel()** compte les valeurs d'un tenseur de paramètres

Source : Documentation PyTorch : torch.nn.Conv2d.

Pooling en PyTorch

Pourquoi : retrouver en code les valeurs de la diapositive « Le pooling », et le pooling global de ResNet-18.

```
import torch
import torch.nn as nn

x = torch.tensor([[1., 3., 2., 0.],
                  [5., 4., 1., 1.],
                  [0., 2., 6., 7.],
                  [1., 1., 8., 3.]]).reshape(1, 1, 4, 4)   # (N, C, H, W)

print(nn.MaxPool2d(kernel_size=2)(x))   # le maximum de chaque zone 2 x 2
# tensor([[[[5., 2.],
#           [2., 8.]]]])
print(nn.AvgPool2d(kernel_size=2)(x))   # la moyenne de chaque zone 2 x 2
# tensor([[[[3.2500, 1.0000],
#           [1.0000, 6.0000]]]])
carte = torch.randn(1, 512, 7, 7)       # la sortie de layer4 de ResNet-18
print(nn.AdaptiveAvgPool2d(1)(carte).shape)   # torch.Size([1, 512, 1, 1])
```

- **MaxPool2d(kernel_size=2)** : zones 2 × 2 ; le pas vaut par défaut la taille de la zone, la carte est divisée par 2
- **AvgPool2d** : même découpage, la moyenne au lieu du maximum
- **AdaptiveAvgPool2d(1)** : une seule zone par canal, quelle que soit la taille de la carte : le pooling global de ResNet-18
- Aucun paramètre : rien à entraîner, rien à geler

Source : Documentation PyTorch : torch.nn.MaxPool2d, AvgPool2d, AdaptiveAvgPool2d.

BatchNorm en PyTorch

Pourquoi : voir les deux modes sur un exemple chiffré.

```
import torch
import torch.nn as nn

torch.manual_seed(0)
bn = nn.BatchNorm2d(3)                    # un gamma et un beta par canal
x = torch.randn(8, 3, 32, 32) * 5 + 10    # un lot : moyenne 10, écart-type 5

bn.train()                                # entraînement : stats du lot
y = bn(x)
print(y.mean().abs() < 1e-3, round(y.std().item(), 3))   # tensor(True) 1.0
print(bn.running_mean)    # tensor([1.0015, 0.9929, 0.9929])
# moyenne mémorisée, mise à jour à chaque lot : 0.9 * 0 + 0.1 * 10 = 1

bn.eval()                                 # évaluation : stats mémorisées
print(round(bn(x).mean().item(), 3))      # 4.853 : sortie non centrée ici
```

- **BatchNorm2d(3)** : un γ et un β par canal, appris
- En **train()** : et à l'initialisation (γ = 1, β = 0), la sortie est centrée réduite
- **running_mean** : la moyenne mobile, qui avance de 10 % vers celle du lot à chaque lot ; après un seul lot, environ 1 au lieu de 10
- En **eval()** : ce sont ces statistiques mémorisées qui servent ; encore loin de celles du lot, d'où une sortie non centrée

Source : Documentation PyTorch : torch.nn.BatchNorm2d.

Un CNN complet en PyTorch

Pourquoi : assembler les briques de la partie 1 : c'est le schéma de la vue d'ensemble, en code.

```
import torch
import torch.nn as nn

def etage(c_in, c_out):    # convolution, BatchNorm, ReLU, puis pooling
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
                         nn.BatchNorm2d(c_out), nn.ReLU(),
                         nn.MaxPool2d(2))

cnn = nn.Sequential(
    etage(3, 32), etage(32, 64), etage(64, 128),   # le backbone
    nn.AdaptiveAvgPool2d(1), nn.Flatten(),         # pooling global
    nn.Linear(128, 37))                            # la tête : 37 races

x = torch.randn(8, 3, 160, 160)                    # un lot de 8 images
for couche in cnn:
    x = couche(x)
    print(type(couche).__name__, tuple(x.shape))
# Sequential (8, 32, 80, 80)          taille divisée par 2
# Sequential (8, 64, 40, 40)
# Sequential (8, 128, 20, 20)
# AdaptiveAvgPool2d (8, 128, 1, 1)    une valeur par canal
# Flatten (8, 128)
# Linear (8, 37)                      un score par race
print(sum(p.numel() for p in cnn.parameters()))   # 98245
```

- **etage** : convolution, BatchNorm, ReLU, puis pooling, les briques de la partie 1
- À chaque étage, la taille est **divisée par 2** et les canaux **doublés**
- **AdaptiveAvgPool2d(1)** et **Linear** : le pooling global et la tête, comme dans ResNet-18
- **98 245** paramètres, contre 11,7 M pour ResNet-18 : même schéma, en beaucoup plus petit

Source : Documentation PyTorch : torch.nn.Sequential.

Bloc résiduel en PyTorch

Pourquoi : passer du schéma au code : c'est le bloc de base de ResNet-18, utilisé au TD.

```python
import torch
import torch.nn as nn

class BlocResiduel(nn.Module):
    def __init__(self, canaux):
        super().__init__()
        self.conv1 = nn.Conv2d(canaux, canaux, 3, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(canaux)
        self.conv2 = nn.Conv2d(canaux, canaux, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(canaux)
        self.relu = nn.ReLU()

    def forward(self, x):
        f = self.relu(self.bn1(self.conv1(x)))
        f = self.bn2(self.conv2(f))        # f = F(x), la correction apprise
        return self.relu(f + x)            # y = ReLU(F(x) + x)

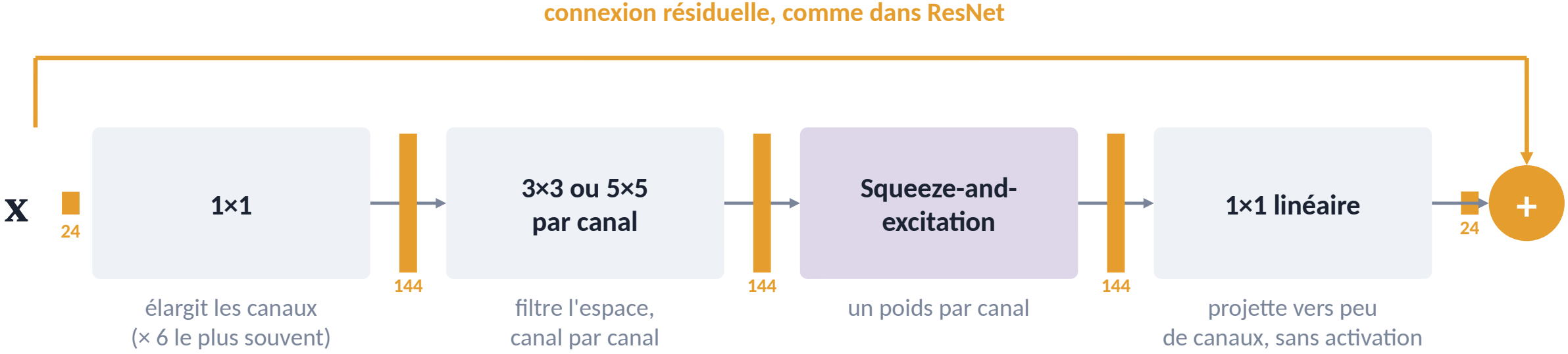
bloc = BlocResiduel(64)
print(bloc(torch.randn(1, 64, 56, 56)).shape)  # torch.Size([1, 64, 56, 56])
```

- Un module PyTorch : **__init__** déclare les couches, **forward** les enchaîne
- **bias=False** : la BatchNorm qui suit apporte déjà un décalage
- Le raccourci tient en **f + x**
- Entrée et sortie de même forme ; sinon, le raccourci passe par une convolution 1 × 1

Source : He et al., CVPR 2016 ; torchvision, BasicBlock (code de resnet18).

Le bloc MBConv

Pourquoi : c'est le bloc d'EfficientNet ; au TD, on dégèle les derniers étages formés de ces blocs.



- **Bottleneck inversé** : les barres orange montrent le nombre de canaux, ici pour un bloc de features[2] d'EfficientNet-B0 (24 → 144 → 24, relevé dans torchvision). Le bottleneck de ResNet va large → étroit → large ; MBConv va étroit → large → étroit
- La connexion résiduelle n'existe que si l'entrée et la sortie ont **la même forme** (même nombre de canaux, même résolution)

Squeeze-and-excitation : un pooling global résume chaque canal en une valeur ; un petit réseau en déduit un poids entre 0 et 1, qui multiplie ce canal. Le réseau renforce ainsi les canaux utiles pour l'image en cours.

Source : Sandler et al., MobileNetV2, CVPR 2018 ; Hu et al., Squeeze-and-Excitation Networks, CVPR 2018 ; Tan et Le, ICML 2019.

Attention en PyTorch

Pourquoi : voir en quelques lignes ce que calcule une couche d'attention.

```python
import torch

N, d_k = 196, 64        # 196 tokens ; requêtes et clés de dimension 64
x = torch.randn(N, d_k)
# Wq, Wk, Wv sont appris en vrai ; ici tirés au hasard, divisés par
# sqrt(d_k) pour garder des valeurs d'ordre 1 (rien à voir avec le softmax)
Wq, Wk, Wv = (torch.randn(d_k, d_k) / d_k**0.5 for _ in range(3))

q, k, v = x @ Wq, x @ Wk, x @ Wv
poids = torch.softmax(q @ k.T / d_k**0.5, dim=-1)  # (196, 196)
sortie = poids @ v                                 # (196, 64)

print(poids.shape)   # torch.Size([196, 196])
print(sortie.shape)  # torch.Size([196, 64])
print(poids.sum(dim=-1)[:3])       # tensor([1.0000, 1.0000, 1.0000])
```

- **q @ k.T** : tous les produits scalaires d'un coup, une matrice 196 × 196
- **softmax(dim=-1)** normalise chaque ligne
- Les scores sont divisés par $\sqrt{d_k}$: le produit scalaire a une variance de l'ordre de d_k, la division la ramène à 1 ; sans elle, pour d_k grand, le softmax donne presque tout le poids à un seul élément
- En pratique : **nn.MultiheadAttention**, plusieurs têtes en parallèle

Source : d2l, section 11.3 ; documentation PyTorch : torch.nn.MultiheadAttention.

Patches et tokens en PyTorch

Pourquoi : voir que le découpage en patches n'est qu'une convolution, codée ainsi dans timm.

```python
import torch
import torch.nn as nn

x = torch.randn(1, 3, 224, 224)
# noyau 16 x 16 et pas de 16 : une projection linéaire par patch
projection = nn.Conv2d(3, 192, kernel_size=16, stride=16)

t = projection(x)                    # (1, 192, 14, 14)
t = t.flatten(2).transpose(1, 2)     # (1, 196, 192) : 196 tokens

cls = nn.Parameter(torch.zeros(1, 1, 192))          # le token [CLS], appris
position = nn.Parameter(torch.zeros(1, 197, 192))   # une position par token
t = torch.cat([cls, t], dim=1) + position
print(t.shape)                       # torch.Size([1, 197, 192])
```

- Une convolution de **noyau 16 et de pas 16** applique la même projection linéaire à chaque patch
- **flatten(2).transpose(1, 2)** : la grille 14 × 14 devient une suite de 196 tokens
- [CLS] et positions : des **nn.Parameter**, appris avec le reste
- Dans timm, cette couche s'appelle **patch_embed.proj** : Conv2d(3, 192, 16, 16) pour DeiT-tiny

Source : Dosovitskiy et al., ICLR 2021 ; bibliothèque timm (pytorch-image-models), PatchEmbed.

Métriques avec scikit-learn

Pourquoi : calculer ces métriques comme au TD (TODO 6).

```python
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score

y_vrai = [0, 0, 1, 1, 2, 2, 2]
y_pred = [0, 1, 1, 1, 2, 0, 2]

print(round(accuracy_score(y_vrai, y_pred), 3))    # 0.714 : 5 sur 7
print(round(f1_score(y_vrai, y_pred, average="macro"), 3))   # 0.7

# lignes : classe réelle ; colonnes : classe prédite
print(confusion_matrix(y_vrai, y_pred))
# [[1 1 0]
#  [0 2 0]
#  [1 0 2]]
```

- **average="macro"** : moyenne des F1 de chaque classe
- Matrice de confusion : **lignes = classe réelle, colonnes = classe prédite** (convention de scikit-learn ; certains ouvrages font l'inverse)
- Ici : une image de la classe 0 prédite 1, une de la classe 2 prédite 0

Source : scikit-learn, section 3.4 : accuracy_score, f1_score, confusion_matrix.

Pour aller plus loin

Pourquoi : des cours en ligne, gratuits, qui reprennent les notions du jour avec plus de détails et des exercices.

Dive into Deep Learning (d2l) d2l.ai	Livre en ligne avec code PyTorch. Chapitre 7 : convolution, pooling ; 8 : BatchNorm, ResNet ; 11 : attention, ViT ; 14.2 : fine-tuning	anglais
CS231n, Stanford cs231n.github.io	Notes de cours sur les CNN (convolution, pooling, normalisation) et sur le transfer learning	anglais
Tutoriel PyTorch : transfer learning docs.pytorch.org	Fine-tuning et feature extraction d'un ResNet-18, pas à pas	anglais
Tutoriel PyTorch : Learn the Basics docs.pytorch.org	Tenseurs, Dataset, nn.Module, autograd, boucle d'entraînement	anglais
Wikipédia : Réseau neuronal convolutif fr.wikipedia.org	Vue d'ensemble des CNN : convolution, pooling, architectures	français
Wikipédia : Vision transformer en.wikipedia.org	Découpage en patches, token [CLS], comparaison avec les CNN	anglais
Fidle (CNRS) fidle.cnrs.fr	Formation au deep learning en vidéo, avec notebooks	français
scikit-learn : Metrics and scoring scikit-learn.org	Accuracy, F1 macro, matrice de confusion, avec exemples	anglais

Références

- Parkhi, Vedaldi, Zisserman et Jawahar, Cats and Dogs (jeu Oxford-IIIT Pet), CVPR 2012.
- Song et Yan, A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects (jeu NEU), Applied Surface Science, 2013.
- Dalal et Triggs, Histograms of Oriented Gradients for Human Detection, CVPR 2005.
- Russakovsky et al., ImageNet Large Scale Visual Recognition Challenge, IJCV 2015.
- Krizhevsky, Sutskever et Hinton, ImageNet Classification with Deep Convolutional Neural Networks (AlexNet), NIPS 2012.
- Simonyan et Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG), ICLR 2015.
- Vaswani et al., Attention Is All You Need, NeurIPS 2017.
- Ioffe et Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, ICML 2015.
- He, Zhang, Ren et Sun, Deep Residual Learning for Image Recognition, CVPR 2016.
- Howard et al., MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications, 2017.
- Sandler et al., MobileNetV2: Inverted Residuals and Linear Bottlenecks, CVPR 2018.
- Hu, Shen et Sun, Squeeze-and-Excitation Networks, CVPR 2018.
- Tan et Le, EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks, ICML 2019.
- Dosovitskiy et al., An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale, ICLR 2021.
- Touvron et al., Training data-efficient image transformers & distillation through attention, ICML 2021.
- Yosinski et al., How transferable are features in deep neural networks?, NIPS 2014.
- Selvaraju et al., Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization, ICCV 2017.
- Zech et al., Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs, PLOS Medicine 2018.
- Zhang, Lipton, Li et Smola, Dive into Deep Learning, Cambridge University Press, 2023 (d2l.ai).

Précisions sur ImageNet : documentation de torchvision (poids IMAGENET1K_V1, une vue) et dépôt officiel de DeiT. Paramètres recomptés à partir des modèles.